

DATA 260 Homework 4

React, MySQL Performance, and Grounded RAG

Priyank Mehta

SJSU ID 019152562

Municipal Transit Incidents

Repository: <https://github.com/priyankmehta5/data260-2562>

Tagged commit: da28c955e4a11c21d367707d6735c829890e715e

This report documents the completed HW4 implementation and the evidence used to verify each required function.

1 Personal Configuration and Submission Files

The system extends the existing municipal transit incident service with a React client, MySQL persistence, server-side sessions, N+1 query measurement and tuning, and a grounded RAG evaluation. All personal values below are derived from SID4 2562 and remain fixed for the assignment.

Configuration	Value
SID4	2562
PORT_BASE	8762
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2
Assigned domain	Municipal transit incidents
Hardware	Intel Core i7-8750H; 15.86 GB RAM; NVIDIA GTX 1050 Ti; Intel UHD 630
Operating system	Windows 10
Local LLM	qwen3:1.7b
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Tagged commit	da28c955e4a11c21d367707d6735c829890e715e

Required repository contents

- reports/hw04/RUN_LOG.txt
- reports/hw04/raw/ (machine-readable experiment and RAG outputs)
- reports/hw04/METRICS.md
- reports/hw04/AI_USE.md
- reports/hw04/report.pdf
- reports/hw04/verification.json
- code/verify_hw04.py
- Tag: hw4

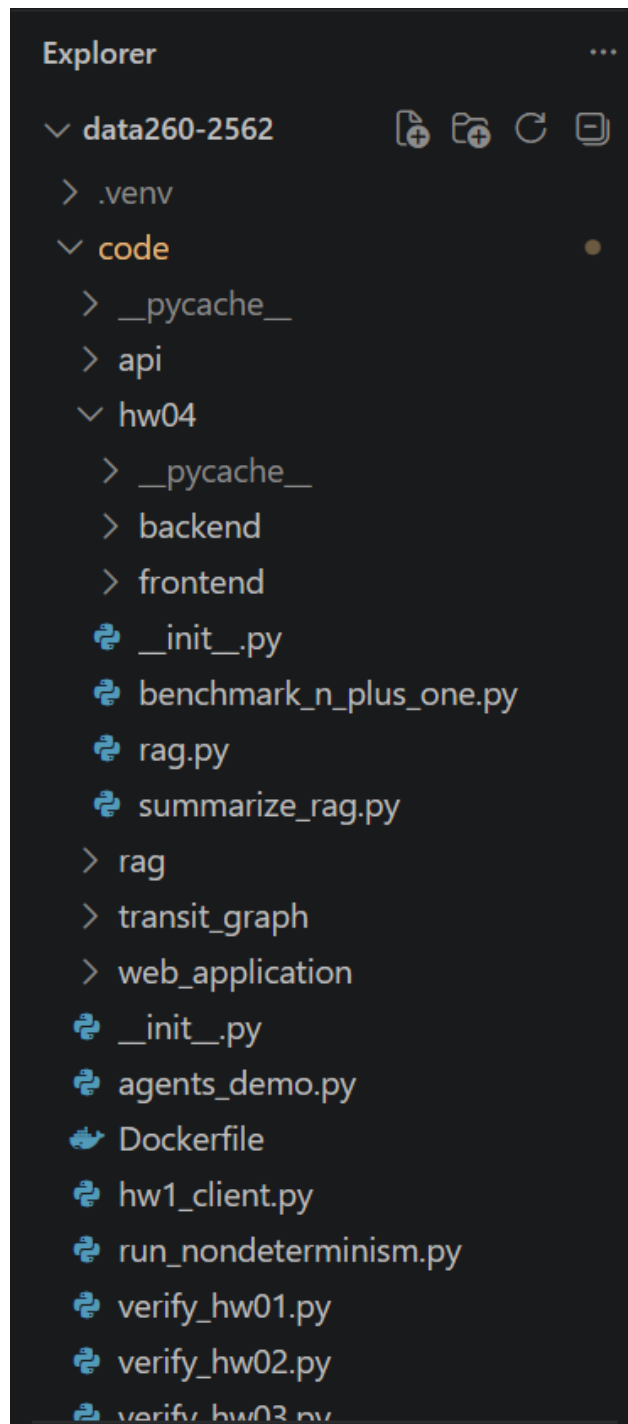


Figure 1. HW4 application code structure in the shared repository.

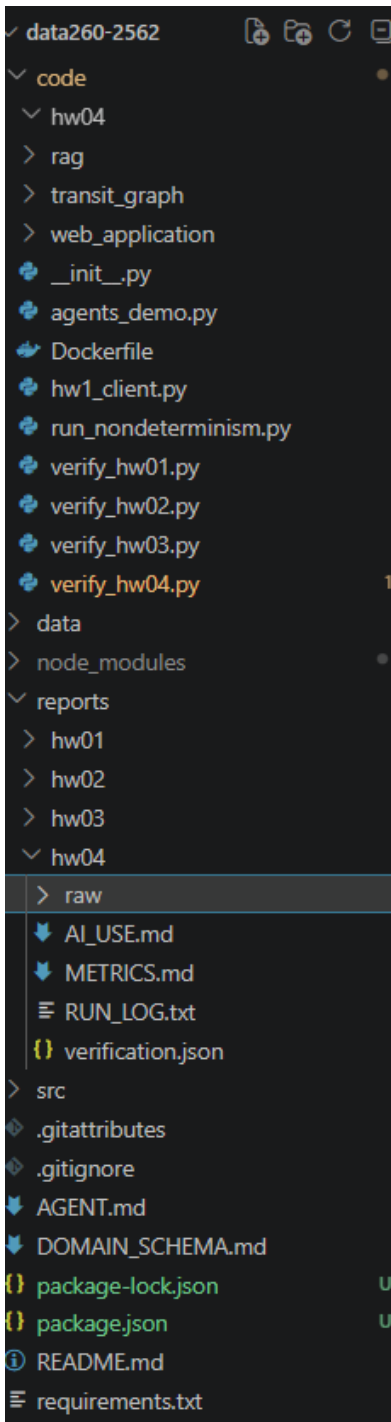


Figure 2. Required reports/hw04 files and repository-level structure.

2 Part 1 React Client

The React client authenticates through the FastAPI backend and uses an HTTP-only session cookie. Protected record operations are hidden behind the authenticated application state. React Router maps the required paths to the Home, CreateRecord, UpdateRecord, and DeleteRecord components.

2.1 Login and authenticated access

```

code > hw04 > frontend > src > Login.jsx > default
1  import { useState } from "react";
2  import api from "../api";
3
4  function Login({ onLogin }) {
5    const [email, setEmail] = useState("student@example.com");
6    const [password, setPassword] = useState("Transit2562!");
7    const [error, setError] = useState("");
8
9    async function handleSubmit(event) {
10     event.preventDefault();
11     setError("");
12
13     try {
14       const response = await api.post("/api/login", {
15         email,
16         password,
17       });
18
19       onLogin(response.data);
20     } catch (requestError) {
21       setError(
22         requestError.response?.data?.detail ||
23         "Login failed"
24       );
25     }
26   }
27
28   return (
29     <main className="login-page">
30       <form className="card login-form" onSubmit={handleSubmit}>
31         <h1>Transit Incident Login</h1>
32
33         <label htmlFor="email">Email</label>
34         <input
35           id="email"
36           type="email"
37           value={email}
38           onChange={(event) => setEmail(event.target.value)}
39           required
40         />
41
42         <label htmlFor="password">Password</label>
43         <input
44           id="password"
45           type="password"
46           value={password}
47           onChange={(event) => setPassword(event.target.value)}
48           required
49         />
50
51         <button type="submit">Log in</button>
52
53         {error && <p className="error">{error}</p>}
54       </form>
55     </main>

```

Figure 3. Login.jsx uses state hooks and submits email and password to /api/login.

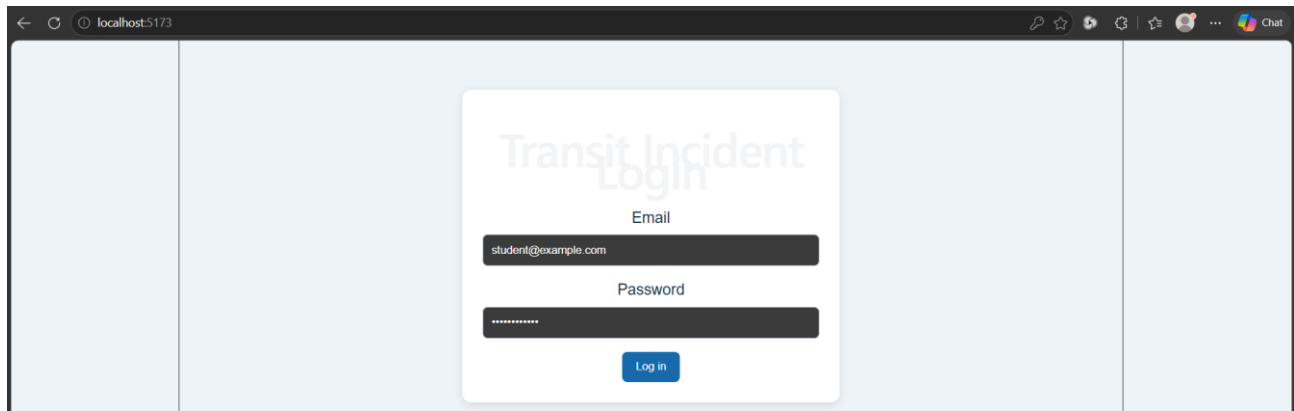


Figure 4. React login page with the required email and password fields and Log in button.

After a successful request, the backend creates a server-side session and the client renders the protected application. The password is not stored in the browser session cookie; only the opaque token is returned as an HTTP-only cookie.

2.2 Home Page

```

code > hw04 > frontend > src > Home.jsx > default
~
5 function Home({ user, onLogout }) {
6   const [incidents, setIncidents] = useState([]);
7   const [error, setError] = useState("");
8
9   useEffect(() => {
10     async function loadIncidents() {
11       try {
12         const response = await api.get("/api/incidents");
13         setIncidents(response.data);
14       } catch {
15         setError("Unable to load incidents");
16       }
17     }
18
19     loadIncidents();
20   }, []);
21
22   return (
23     <main className="page">
24       <header className="page-header">
25         <div>
26           <h1>Municipal Transit Incidents</h1>
27           <p>Logged in as {user.name}</p>
28         </div>
29
30         <button onClick={onLogout}>Log out</button>
31       </header>
32
33       <section className="card">
34         <div className="section-heading">
35           <h2>Incident Records</h2>
36           <Link className="button-link" to="/create">
37             Create incident
38           </Link>
39         </div>
40
41         {incidents.length === 0 && (
42           <p>No incident records are available.</p>
43         )}
44
45         {incidents.map((incident) => (
46           <article className="incident" key={incident.id}>
47             <div>
48               <h3>
49                 #{incident.id} - {incident.title}
50               </h3>
51               <p>{incident.description}</p>
52             </div>
53
54             <div className="actions">
55               <Link
56                 className="button-link"
57                 to={` /update/${incident.id}`}
58               >
59                 Update
60               </Link>
61
62               <Link
63                 className="button-link danger"
64                 to={` /delete/${incident.id}`}
65               >
66                 Delete
67               </Link>
68             </div>
69           </article>
70         ))}
71

```

Figure 5. Home.jsx loads incidents with useEffect and renders update and delete controls.

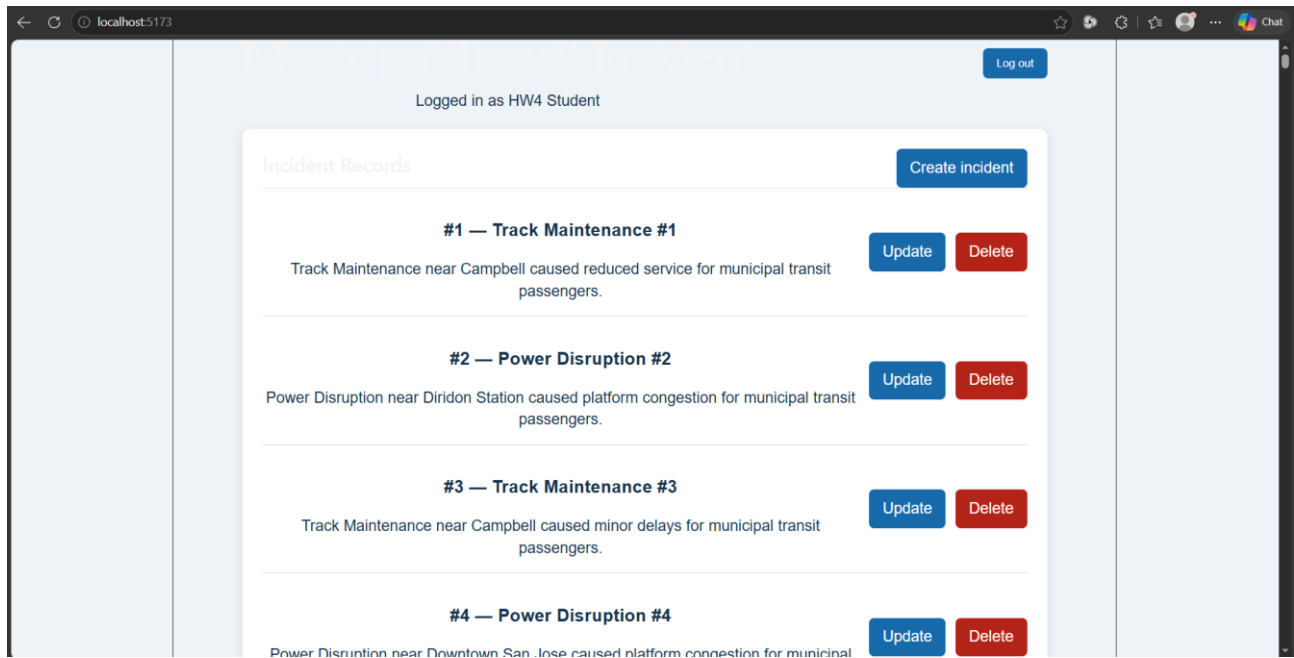


Figure 6. Authenticated root route displaying municipal transit incident records.

2.3 Add a New Record

```

code > hw04 > frontend > src > CreateRecord.jsx > default
1 import { useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import api from "../api";
4
5 function CreateRecord({ pageTitle }) {
6   const navigate = useNavigate();
7   const [title, setTitle] = useState("");
8   const [description, setDescription] = useState("");
9   const [error, setError] = useState("");
10
11   async function handleSubmit(event) {
12     event.preventDefault();
13
14     try {
15       await api.post("/api/incidents", {
16         title,
17         description,
18       });
19       navigate("/");
20     } catch (requestError) {
21       setError(
22         requestError.response?.data?.detail ||
23         "Unable to create incident"
24       );
25     }
26   }
27
28   return (
29     <main className="page">
30       <form className="card" onSubmit={handleSubmit}>
31         <h1>{pageTitle}</h1>
32
33         <label htmlFor="title">Incident title</label>
34         <input
35           id="title"
36           value={title}
37           onChange={(event) => setTitle(event.target.value)}
38           required
39         />
40
41         <label htmlFor="description">Description</label>
42         <textarea
43           id="description"
44           value={description}
45           onChange={(event) => setDescription(event.target.value)}
46           required
47         />
48
49         <div className="actions">
50           <button type="submit">Create</button>
51           <button
52             type="button"
53             className="secondary"
54             onClick={() => navigate("/")}
55           >
56             Cancel
57           </button>
58         </div>
59
60         {error && <p className="error">{error}</p>}
61       </form>
62     </main>
63   );
64 }

```

Figure 7. CreateRecord.jsx accepts a pageTitle prop, manages form state, posts the record, and navigates to the home route.

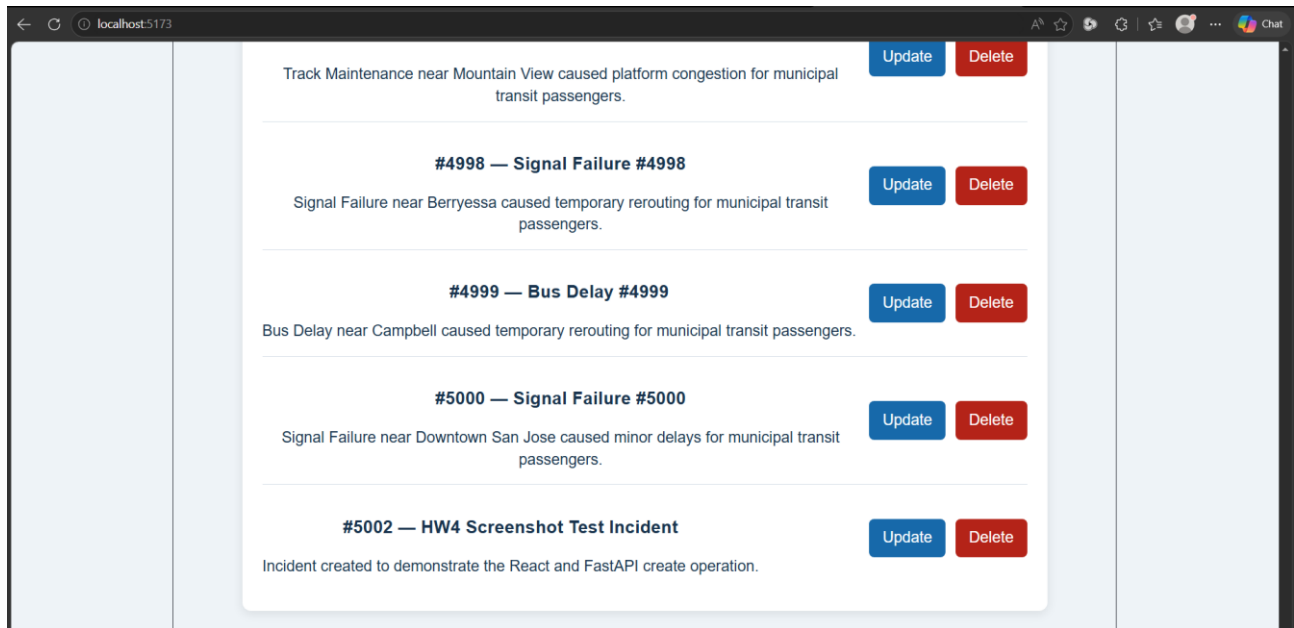


Figure 8. Newly created incident displayed after redirect to the home page.

2.4 Update Record

```

code > hw04 > frontend > src > UpdateRecord.jsx > UpdateRecord > handleSub
5  function UpdateRecord({ pageTitle }) {
6    const { id } = useParams();
7    const navigate = useNavigate();
8    const [title, setTitle] = useState("");
9    const [description, setDescription] = useState("");
10   const [error, setError] = useState("");
11
12   useEffect(() => {
13     async function loadIncident() {
14       try {
15         const response = await api.get(`/api/incidents/${id}`);
16         setTitle(response.data.title);
17         setDescription(response.data.description);
18       } catch {
19         setError("Unable to load incident");
20       }
21     }
22
23     loadIncident();
24   }, [id]);
25
26   async function handleSubmit(event) {
27     event.preventDefault();
28
29     try {
30       await api.put(`/api/incidents/${id}`, {
31         title,
32         description,
33       });
34
35       navigate("/");
36     } catch (requestError) {
37       setError(
38         requestError.response?.data?.detail ||
39         "Unable to update incident"
40       );
41     }
42   }
43
44   return (
45     <main className="page">
46       <form className="card" onSubmit={handleSubmit}>
47         <h1>{pageTitle}</h1>
48
49         <label htmlFor="title">Incident title</label>
50         <input
51           id="title"
52           value={title}
53           onChange={(event) => setTitle(event.target.value)}
54           required
55         />
56
57         <label htmlFor="description">Description</label>
58         <textarea
59           id="description"
60           value={description}
61           onChange={(event) => setDescription(event.target.value)}
62           required
63         />
64
65         <div className="actions">
66           <button type="submit">Save changes</button>
67           <button
68             type="button"
69             className="secondary"
70           >

```

Figure 9. UpdateRecord.jsx loads the selected record, submits PUT data, and redirects after success.

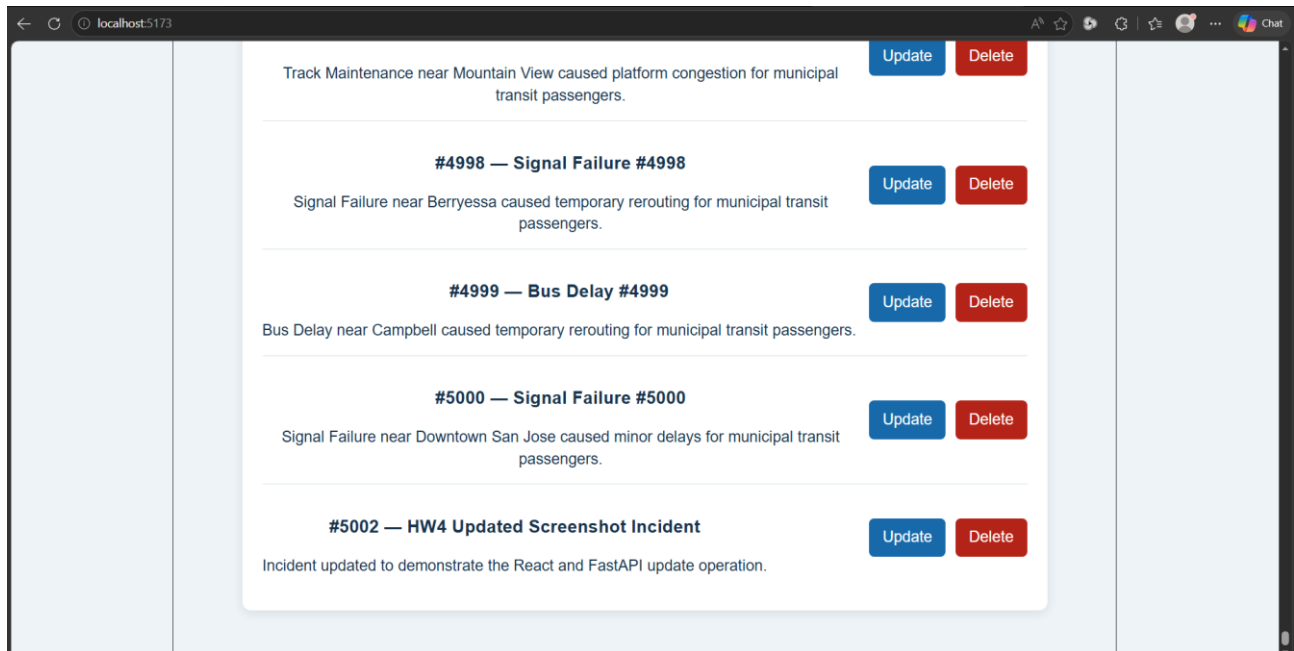


Figure 10. Updated incident persisted and displayed on the home page.

2.5 Delete Record

```

code > hw04 > frontend > src > DeleteRecord.jsx > default
1  import { useEffect, useState } from "react";
2  import { useNavigate, useParams } from "react-router-dom";
3  import api from "../api";
4
5  function DeleteRecord({ pageTitle }) {
6    const { id } = useParams();
7    const navigate = useNavigate();
8    const [incident, setIncident] = useState(null);
9    const [error, setError] = useState("");
10
11    useEffect(() => {
12      async function loadIncident() {
13        try {
14          const response = await api.get(`/api/incidents/${id}`);
15          setIncident(response.data);
16        } catch {
17          setError("Unable to load incident");
18        }
19      }
20
21      loadIncident();
22    }, [id]);
23
24    async function handleDelete() {
25      try {
26        await api.delete(`/api/incidents/${id}`);
27        navigate("/");
28      } catch (requestError) {
29        setError(
30          requestError.response?.data?.detail ||
31          "Unable to delete incident"
32        );
33      }
34    }
35
36    return (
37      <main className="page">
38        <section className="card">
39          <h1>{pageTitle}</h1>
40
41          {incident && (
42            <>
43              <h2>{incident.title}</h2>
44              <p>{incident.description}</p>
45
46              <div className="actions">
47                <button
48                  className="danger"
49                  onClick={handleDelete}>
50                >
51                Confirm delete
52              </button>
53
54              <button
55                className="secondary"
56                onClick={() => navigate("/")}>
57            >
58            Cancel
59          </button>
60        </div>
61      </>
62    )}
63
64    {error && <p className="error">{error}</p>}
65  </section>
66 </main>
67 );

```

Figure 11. DeleteRecord.jsx loads the selected incident and sends the delete request.

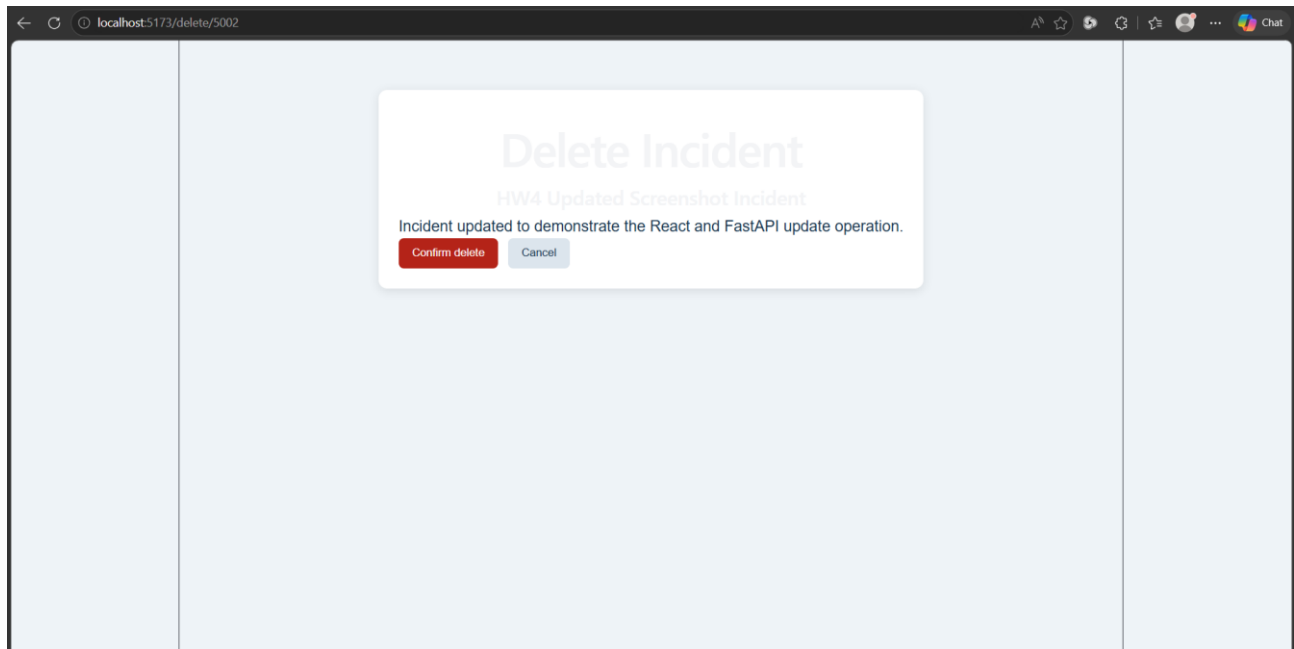


Figure 12. Delete confirmation page with domain-appropriate controls.

2.6 Props, hooks, and routing

```

code > hw04 > frontend > src > App.jsx > default
17  function App() {
49      return (
50          <BrowserRouter>
51              <Routes>
52                  <Route
53                      path="/"
54                      element={
55                          <Home user={user} onLogout={handleLogout} />
56                      }
57                  />
58
59                  <Route
60                      path="/create"
61                      element={
62                          <CreateRecord pageTitle="Create Incident" />
63                      }
64                  />
65
66                  <Route
67                      path="/update/:id"
68                      element={
69                          <UpdateRecord pageTitle="Update Incident" />
70                      }
71                  />
72
73                  <Route
74                      path="/delete/:id"
75                      element={
76                          <DeleteRecord pageTitle="Delete Incident" />
77                      }
78                  />
79
80                  <Route path="*" element={<Navigate to="/" replace />} />
81              </Routes>
82          </BrowserRouter>
83      );
84  }
85
86  export default App;

```

Figure 13. App.jsx maps /, /create, /update/:id, and /delete/:id with React Router and passes component props.

The client uses `useState` for form and authentication state, `useEffect` for data loading, `useParams` for record identifiers, and `useNavigate` for post-operation redirects. The route parameters extend the required `/update` and `/delete` routes so the selected database record can be addressed unambiguously.

3 Part 2 MySQL Persistence and Server Side Sessions

3.1 Database connection and schema

```

code > hw04 > backend > database.py > get_db
1  import os
2  from pathlib import Path
3
4  from dotenv import load_dotenv
5  from sqlalchemy import URL, create_engine
6  from sqlalchemy.orm import declarative_base, sessionmaker
7
8
9  load_dotenv(Path(__file__).with_name(".env"))
10
11  DATABASE_URL = URL.create(
12      drivename="mysql+pymysql",
13      username=os.getenv("MYSQL_USER", "root"),
14      password=os.getenv("MYSQL_PASSWORD"),
15      host=os.getenv("MYSQL_HOST", "127.0.0.1"),
16      port=int(os.getenv("MYSQL_PORT", "3306")),
17      database=os.getenv("MYSQL_DATABASE", "s2562_rel"),
18  )
19
20  engine = create_engine(
21      DATABASE_URL,
22      pool_pre_ping=True,
23  )
24
25  db_session_basedede26 = sessionmaker(
26      autocommit=False,
27      autoflush=False,
28      bind=engine,
29  )
30
31  Base = declarative_base()
32
33
34  def get_db():
35      database = db_session_basedede26()
36
37      try:
38          yield database
39      finally:
40          database.close()

```

Figure 14. SQLAlchemy MySQL connection and the strictly required `db_session_basedede26` variable.

```

database.py × models.py × init_db.py × schemas.py ×
code > hw04 > backend > models.py > Session
9 class Incident(Base):
10     __tablename__ = "incidents"
11
12     id: Mapped[int] = mapped_column(
13         Integer,
14         primary_key=True,
15         autoincrement=True,
16     )
17     title: Mapped[str] = mapped_column(String(100))
18     description: Mapped[str] = mapped_column(Text)
19
20
21 class IncidentNote(Base):
22     __tablename__ = "incident_notes"
23
24     id: Mapped[int] = mapped_column(
25         Integer,
26         primary_key=True,
27         autoincrement=True,
28     )
29     incident_id: Mapped[int] = mapped_column(Integer)
30     note: Mapped[str] = mapped_column(String(255))
31
32
33 class User(Base):
34     __tablename__ = "users"
35
36     id: Mapped[int] = mapped_column(
37         Integer,
38         primary_key=True,
39         autoincrement=True,
40     )
41     name: Mapped[str] = mapped_column(String(100))
42     email: Mapped[str] = mapped_column(
43         String(255),
44         unique=True,
45         nullable=False,
46     )
47     password_hash: Mapped[str] = mapped_column(
48         String(255),
49         nullable=False,
50     )
51
52
53 class Session(Base):
54     __tablename__ = "sessions"
55
56     id: Mapped[int] = mapped_column(
57         Integer,
58         primary_key=True,
59         autoincrement=True,
60     )
61     token: Mapped[str] = mapped_column(
62         String(128),
63         unique=True,
64         nullable=False,
65     )
66     user_id: Mapped[int] = mapped_column(
67         Integer,
68         nullable=False,
69     )
70     created_at: Mapped[datetime] = mapped_column(
71         DateTime,
72         nullable=False,
73     )
74     expires_at: Mapped[datetime] = mapped_column(
75         DateTime,

```

Figure 15. SQLAlchemy models for incidents, incident_notes, users, and sessions.

The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'Navigator' pane with sections for 'MANAGEMENT' (Server Status, Client Connections, Users and Privileges, Status and System Variables, Data Export, Data Import/Restore), 'INSTANCE' (Startup / Shutdown, Server Logs, Options File), and 'PERFORMANCE' (Dashboard, Performance Reports, Performance Schema Setup). The 'Administration' and 'Schemas' tabs are visible at the bottom of the sidebar.

The main window displays a query editor with the following SQL code:

```

1 USE s2562_rel;
2
3 SELECT COUNT(*) AS incident_count
4 FROM incidents;
5
6 SELECT COUNT(*) AS related_note_count
7 FROM incident_notes;

```

The 'Result Grid' tab is active, showing a single row with the value '5000' under the column 'incident_count'. The 'Output' pane at the bottom shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
13	21:12:39	SELECT * FROM incidents LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
14	21:12:56	SELECT COUNT(*) AS incident_count FROM incidents LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
15	21:13:13	SELECT COUNT(*) AS related_note_count FROM incident_notes LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
16	21:13:25	SELECT COUNT(*) AS incident_count FROM incidents LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
17	23:40:01	SELECT COUNT(*) AS related_note_count FROM incident_notes LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
18	23:40:10	SELECT COUNT(*) AS incident_count FROM incidents LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

Figure 16. MySQL Workbench confirms 5,000 incident rows in s2562_rel.

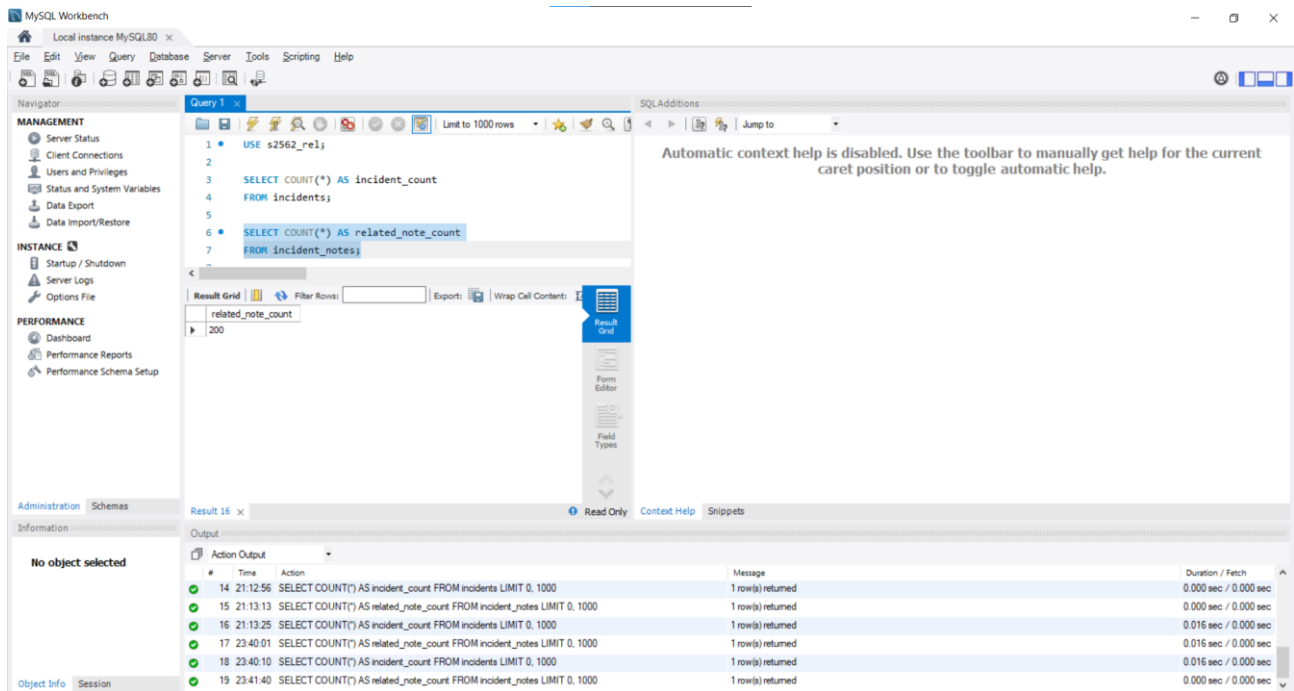


Figure 17. MySQL Workbench confirms 200 related incident-note rows.

The s2562_rel database uses auto-incremented primary keys for incidents and users. The users table enforces a unique email address. The sessions table stores an opaque token, user identifier, creation time, and expiration time.

3.2 Login and session enforcement

```

code > hw04 > backend > main.py > login
87 @app.post("/api/login", response_model=UserResponse)
88 def login(
89     credentials: LoginRequest,
90     response: Response,
91     database: DatabaseSession = Depends(get_db),
92 ):
93     user = (
94         database.query(User)
95         .filter(User.email == credentials.email.strip().lower())
96         .first()
97     )
98
99     password_valid = (
100         user is not None
101         and bcrypt.checkpw(
102             credentials.password.encode("utf-8"),
103             user.password_hash.encode("utf-8"),
104         )
105     )
106
107     if not password_valid:
108         raise HTTPException(
109             status_code=status.HTTP_401_UNAUTHORIZED,
110             detail="Invalid email or password",
111         )
112
113     token = secrets.token_urlsafe(48)
114     expires_at = datetime.utcnow() + timedelta(hours=8)
115
116     database.add(
117         Session(
118             token=token,
119             user_id=user.id,
120             created_at=datetime.utcnow(),
121             expires_at=expires_at,
122         )
123     )
124     database.commit()
125
126     response.set_cookie(
127         key="session_token",
128         value=token,
129         httponly=True,
130         samesite="lax",
131         secure=False,
132         max_age=8 * 60 * 60,
133     )
134
135     return user
136
137
138 @app.post("/api/logout", status_code=204)
139 def logout(
140     request: Request,
141     response: Response,

```

Figure 18. Login endpoint verifies the password, creates a server-side session token, and sets an HTTP-only cookie.

```

code > hw04 > backend > main.py > login > database
44 def get_current_user(
45     request: Request,
46     database: DatabaseSession = Depends(get_db),
47 ):
48     token = request.cookies.get("session_token")
49
50     if not token:
51         raise HTTPException(
52             status_code=status.HTTP_401_UNAUTHORIZED,
53             detail="Login required",
54         )
55
56     stored_session = (
57         database.query(Session)
58         .filter(
59             Session.token == token,
60             Session.expires_at > datetime.utcnow(),
61         )
62         .first()
63     )
64
65     if stored_session is None:
66         raise HTTPException(
67             status_code=status.HTTP_401_UNAUTHORIZED,
68             detail="Invalid or expired session",
69         )
70
71     user = database.get(User, stored_session.user_id)
72
73     if user is None:
74         raise HTTPException(
75             status_code=status.HTTP_401_UNAUTHORIZED,
76             detail="User not found",
77         )
78
79     return user
80
81
82 @app.get("/")
83 def root():
84     return {"message": "Hw4 API is running"}
85
86
87 @app.post("/api/login", response_model=UserResponse)
88 def login(
89     credentials: LoginRequest,
90     response: Response,
91     database: DatabaseSession = Depends(get_db),
92 ):
93     user = (
94         database.query(User)
95         .filter(User.email == credentials.email.strip().lower())
96         .first()
97     )
98

```

Figure 19. Protected-route dependency validates the cookie token against the sessions table and expiration time.

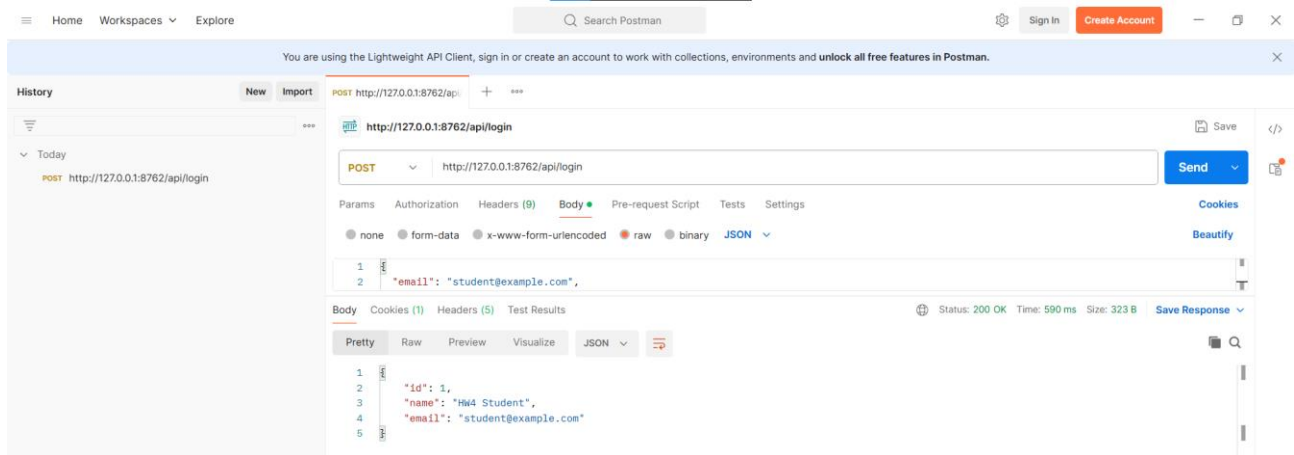


Figure 20. Postman login returns HTTP 200 and receives the session cookie.

3.3 CRUD API

```

database.py X models.py X init_db.py X schemas.py X main.py ...\backen
code > hw04 > backend > main.py > get_incidents
162
163 @app.get("/api/incidents", response_model=list[IncidentResponse])
164 def get_incidents(
165     database: DatabaseSession = Depends(get_db),
166     user: User = Depends(get_current_user),
167 ):
168     return database.query(Incident).order_by(Incident.id).all()
169
170
171 @app.get(
172     "/api/incidents/{incident_id}",
173     response_model=IncidentResponse,
174 )
175 def get_incident(
176     incident_id: int,
177     database: DatabaseSession = Depends(get_db),
178     user: User = Depends(get_current_user),
179 ):
180     incident = database.get(Incident, incident_id)
181
182     if incident is None:
183         raise HTTPException(404, "Incident not found")
184
185     return incident
186
187
188 @app.post(
189     "/api/incidents",
190     response_model=IncidentResponse,
191     status_code=201,
192 )
193 def create_incident(
194     data: IncidentCreate,
195     database: DatabaseSession = Depends(get_db),
196     user: User = Depends(get_current_user),
197 ):
198     incident = Incident(
199         title=data.title.strip(),
200         description=data.description.strip(),
201     )
202
203     database.add(incident)
204     database.commit()
205     database.refresh(incident)
206     return incident
207
208
209 @app.put(
210     "/api/incidents/{incident_id}",
211     response_model=IncidentResponse,
212 )
213 def update_incident(
214     incident_id: int,
215     data: IncidentUpdate,
216     database: DatabaseSession = Depends(get_db),

```

Figure 21. Protected GET, GET-by-ID, and POST incident endpoints.

The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8762/api/incidents` successfully executed. The response is a JSON array of three incident records, and the status is 200 OK.

Request:

- Method: GET
- URL: `http://127.0.0.1:8762/api/incidents`

Response:

```
1 {
2   {
3     "title": "Track Maintenance #1",
4     "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
5     "id": 1
6   },
7   {
8     "title": "Power Disruption #2",
9     "description": "Power Disruption near Diridon Station caused platform congestion for municipal transit passengers.",
10    "id": 2
11  },
12  {
13    "title": "Track Maintenance #3",
14    "description": "Track Maintenance near Campbell caused minor delays for municipal transit passengers.",
15    "id": 3
16  },
17 }
```

Status: 200 OK, Time: 610 ms, Size: 738.61 KB

Figure 22. Postman GET returns all incident records with HTTP 200.

The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8762/api/incidents/1`. The response is a JSON object with the following structure:

```
{
  "title": "Track Maintenance #1",
  "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
  "id": 1
}
```

The status bar indicates a 200 OK response with a time of 25 ms and a size of 270 B. The response is displayed in the 'Body' tab, which is currently set to 'Pretty' view.

History on the left shows the following requests:

- GET `http://127.0.0.1:8762/api/incidents/1`
- GET `http://127.0.0.1:8762/api/incidents`
- POST `http://127.0.0.1:8762/api/login`

At the bottom, there is a message: "Create collections in Postman. Use collections to save your requests and share them with others. Create a Collection".

Figure 23. Postman GET by ID returns incident 1 with HTTP 200.

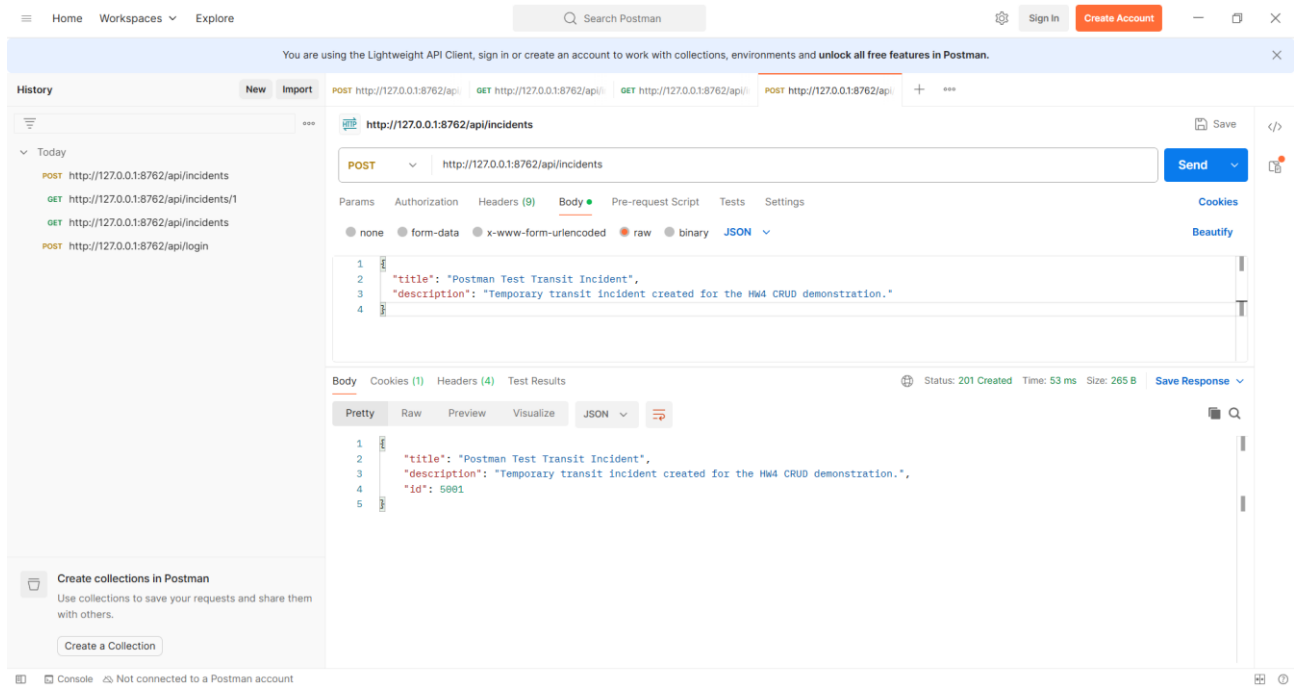


Figure 24. Postman POST creates incident 5001 and returns HTTP 201.

```

code > hw04 > backend > main.py > update_incident
213 def update_incident(
214     incident_id: int,
215     data: IncidentUpdate,
216     database: DatabaseSession = Depends(get_db),
217     user: User = Depends(get_current_user),
218 ):
219     incident = database.get(Incident, incident_id)
220
221     if incident is None:
222         raise HTTPException(404, "Incident not found")
223
224     incident.title = data.title.strip()
225     incident.description = data.description.strip()
226
227     database.commit()
228     database.refresh(incident)
229     return incident
230
231
232 @app.delete(
233     "/api/incidents/{incident_id}",
234     status_code=204,
235 )
236 def delete_incident(
237     incident_id: int,
238     database: DatabaseSession = Depends(get_db),
239     user: User = Depends(get_current_user),
240 ):
241     incident = database.get(Incident, incident_id)
242
243     if incident is None:
244         raise HTTPException(404, "Incident not found")
245
246     database.delete(incident)
247     database.commit()
248     return Response(status_code=204)
249
250 @app.get("/api/performance/incidents-naive")
251 def get_incidents_naive(
252     response: Response,
253     page_size: int = 10,
254     database: DatabaseSession = Depends(get_db),
255     user: User = Depends(get_current_user),
256 ):
257     if page_size not in {10, 50, 200}:
258         raise HTTPException(
259             status_code=400,
260             detail="page_size must be 10, 50, or 200",
261         )
262
263     started = time.perf_counter()
264
265     incidents = (
266         database.query(Incident)
267         .order_by(Incident.id)

```

Figure 25. Protected PUT and DELETE endpoints commit changes through SQLAlchemy.

The screenshot shows the Postman interface with a PUT request configured to update incident 5001. The request URL is `http://127.0.0.1:8762/api/incidents/5001`. The request body is a JSON object: `{ "title": "Updated Postman Transit Incident", "description": "Transit incident successfully updated through the FastAPI and MySQL backend." }`. The response status is 200 OK, with a time of 75 ms and a size of 273 B. The response body is a JSON object: `{ "title": "Updated Postman Transit Incident", "description": "Transit incident successfully updated through the FastAPI and MySQL backend.", "id": 5001 }`.

History: Today
PUT `http://127.0.0.1:8762/api/incidents/5001`
POST `http://127.0.0.1:8762/api/incidents`
GET `http://127.0.0.1:8762/api/incidents/1`
GET `http://127.0.0.1:8762/api/incidents`
POST `http://127.0.0.1:8762/api/login`

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

Console: Not connected to a Postman account

Figure 26. Postman PUT updates incident 5001 and returns HTTP 200.

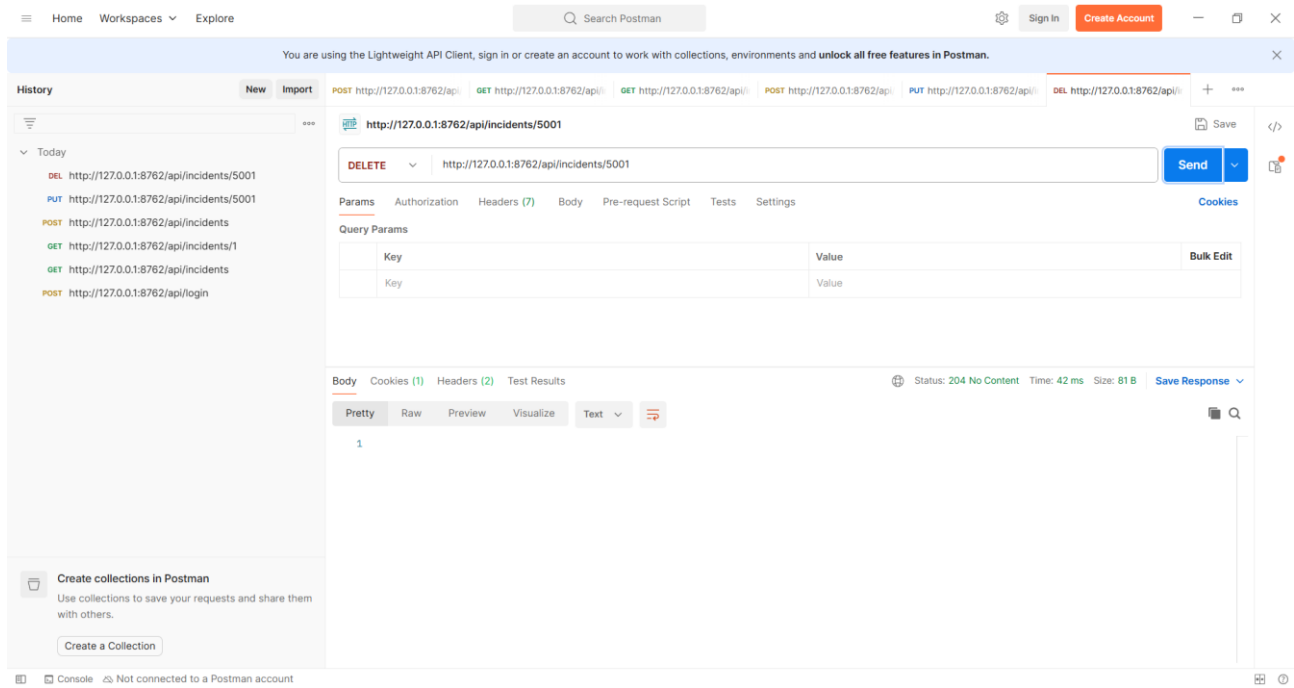


Figure 27. Postman DELETE removes incident 5001 and returns HTTP 204.

4 Part 3 N Plus One Measurement and Query Tuning

4.1 Deterministic seed data

```

code > hw04 > backend > seed_data.py > ...
 9  SEED = 2562
10  INCIDENT_COUNT = 5000
11  NOTE_COUNT = 200
12
13  INCIDENT_TYPES = [
14      "Signal Failure",
15      "Bus Delay",
16      "Track Maintenance",
17      "Station Closure",
18      "Power Disruption",
19      "Vehicle Breakdown",
20      "Service Interruption",
21  ]
22
23  LOCATIONS = [
24      "Santa Clara",
25      "Downtown San Jose",
26      "Mountain View",
27      "Milpitas",
28      "Berryessa",
29      "Campbell",
30      "Diridon Station",
31  ]
32
33  IMPACTS = [
34      "minor delays",
35      "major delays",
36      "temporary rerouting",
37      "reduced service",
38      "platform congestion",
39  ]
40
41
42  def seed_database():
43      random_generator = random.Random(SEED)
44      database = db_session_basede26()
45
46      try:
47          database.query(IncidentNote).delete()
48          database.query(Incident).delete()
49          database.commit()
50
51          database.execute(
52              text("ALTER TABLE incident_notes AUTO_INCREMENT = 1")
53          )
54          database.execute(
55              text("ALTER TABLE incidents AUTO_INCREMENT = 1")
56          )
57
58          incidents = []
59
60          for number in range(1, INCIDENT_COUNT + 1):
61              incident_type = random_generator.choice(
62                  INCIDENT_TYPES
63              )

```

Figure 28. Seed generator uses SEED 2562 and creates 5,000 incidents and 200 related notes.

Figures 16 and 17 provide the corresponding database output. The related note rows reference incident IDs so both the naive and optimized list endpoints can return the same nested record shape.

4.2 Naive and optimized endpoints

```

> hw04 > backend > main.py > get_incidents_optimized

@app.get("/api/performance/incidents-naive")
def get_incidents_naive(
    response: Response,
    page_size: int = 10,
    database: DatabaseSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    if page_size not in {10, 50, 200}:
        raise HTTPException(
            status_code=400,
            detail="page_size must be 10, 50, or 200",
        )

    started = time.perf_counter()

    incidents = (
        database.query(Incident)
        .order_by(Incident.id)
        .limit(page_size)
        .all()
    )

    records = []

    for incident in incidents:
        notes = (
            database.query(IncidentNote)
            .filter(IncidentNote.incident_id == incident.id)
            .all()
        )

        records.append(
            {
                "id": incident.id,
                "title": incident.title,
                "description": incident.description,
                "notes": [
                    {
                        "id": note.id,
                        "note": note.note,
                    }
                    for note in notes
                ],
            }
        )

    latency_ms = round(
        (time.perf_counter() - started) * 1000,
        3,
    )
    sql_queries = 1 + len(incidents)

    response.headers["X-SQL-Queries"] = str(sql_queries)
    response.headers["X-Retrieval-Latency-MS"] = str(
        latency_ms
    )

```

Figure 29. Naive endpoint performs one incident query plus one note query per returned incident.

```

code > hw04 > backend > main.py > get_incidents_optimized
316 @app.get("/api/performance/incidents-optimized")
317 def get_incidents_optimized(
318     response: Response,
319     page_size: int = 10,
320     database: DatabaseSession = Depends(get_db),
321     user: User = Depends(get_current_user),
322 ):
323     if page_size not in {10, 50, 200}:
324         raise HTTPException(
325             status_code=400,
326             detail="page_size must be 10, 50, or 200",
327         )
328
329     started = time.perf_counter()
330
331     incident_page = (
332         database.query(Incident.id)
333         .order_by(Incident.id)
334         .limit(page_size)
335         .subquery()
336     )
337
338     rows = (
339         database.query(Incident, IncidentNote)
340         .join(
341             incident_page,
342             Incident.id == incident_page.c.id,
343         )
344         .outerjoin(
345             IncidentNote,
346             Incident.id == IncidentNote.incident_id,
347         )
348         .order_by(Incident.id)
349         .all()
350     )
351
352     records_by_id = {}
353
354     for incident, note in rows:
355         if incident.id not in records_by_id:
356             records_by_id[incident.id] = {
357                 "id": incident.id,
358                 "title": incident.title,
359                 "description": incident.description,
360                 "notes": [],
361             }
362
363         if note is not None:
364             records_by_id[incident.id]["notes"].append(
365                 {
366                     "id": note.id,
367                     "note": note.note,
368                 }
369             )
370

```

Figure 30. Optimized endpoint joins incidents and notes in one SQL query.

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History: Today
GET http://127.0.0.1:8762/api/performance/incidents-naive...
GET http://127.0.0.1:8762/api/incidents/5001
DEL http://127.0.0.1:8762/api/incidents/5001
PUT http://127.0.0.1:8762/api/incidents/5001
POST http://127.0.0.1:8762/api/incidents
GET http://127.0.0.1:8762/api/incidents/1
GET http://127.0.0.1:8762/api/incidents
POST http://127.0.0.1:8762/api/login

GET http://127.0.0.1:8762/api/performance/incidents-naive?page_size=10

Params: Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ page_size	10	
Key	Value	

Body Cookies (1) Headers (6) Test Results Status: 200 OK Time: 64 ms Size: 1.81 KB Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "version": "naive",
3   "page_size": 10,
4   "sql_queries": 11,
5   "retrieval_latency_ms": 46.887,
6   "records": [
7     {
8       "id": 1,
9       "title": "Track Maintenance #1",
10      "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
11      "notes": []
12    },
13    {
14      "id": 2,
15      "title": "Power Disruption #2",
16      "description": "Power Disruption near Diridon Station caused platform congestion for municipal transit passengers.",
17      "notes": []
18    }
19  ]
20 }
```

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

Console Not connected to a Postman account

Figure 31. Postman naive endpoint at page size 10 returns 10 records and 11 SQL queries.

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- GET http://127.0.0.1:8762/api/performance/incidents-o...
- GET http://127.0.0.1:8762/api/performance/incidents-n...
- GET http://127.0.0.1:8762/api/incidents/5001
- DEL http://127.0.0.1:8762/api/incidents/5001
- PUT http://127.0.0.1:8762/api/incidents/5001
- POST http://127.0.0.1:8762/api/incidents
- GET http://127.0.0.1:8762/api/incidents/1
- GET http://127.0.0.1:8762/api/incidents
- POST http://127.0.0.1:8762/api/login

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

http://127.0.0.1:8762/api/performance/incidents-optimized?page_size=10

GET http://127.0.0.1:8762/api/performance/incidents-optimized?page_size=10

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
page_size	10	
Key	Value	

Body Cookies (1) Headers (6) Test Results Status: 200 OK Time: 41 ms Size: 1.81 KB Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "version": "optimized",
3   "page_size": 10,
4   "sql_queries": 1,
5   "retrieval_latency_ms": 17.779,
6   "records": [
7     {
8       "id": 1,
9       "title": "Track Maintenance #1",
10      "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
11      "notes": []
12    },
13    {
14      "id": 2,
15      "title": "Power Disruption #2",
16      "description": "Power Disruption near Diridon Station caused platform congestion for municipal transit passengers.",
17      "notes": []
18    }
19  ]
20 }
```

Console Not connected to a Postman account

Figure 32. Postman optimized endpoint at page size 10 returns 10 records and 1 SQL query.

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- GET http://127.0.0.1:8762/api/performance/incidents-naive...
- GET http://127.0.0.1:8762/api/performance/incidents-naive...
- GET http://127.0.0.1:8762/api/performance/incidents-naive...
- GET http://127.0.0.1:8762/api/incidents/5001
- DEL http://127.0.0.1:8762/api/incidents/5001
- PUT http://127.0.0.1:8762/api/incidents/5001
- POST http://127.0.0.1:8762/api/incidents
- GET http://127.0.0.1:8762/api/incidents/1
- GET http://127.0.0.1:8762/api/incidents
- POST http://127.0.0.1:8762/api/login

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

http://127.0.0.1:8762/api/performance/incidents-naive?page_size=50

GET http://127.0.0.1:8762/api/performance/incidents-naive?page_size=50

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ page_size	50	
Key	Value	

Body Cookies (1) Headers (6) Test Results Status: 200 OK Time: 147 ms Size: 814 KB Save Response

Pretty Raw Preview Visualize JSON

```
1
2  {
3    "version": "naive",
4    "page_size": 50,
5    "sql_queries": 51,
6    "retrieval_latency_ms": 122.891,
7    "records": [
8      {
9        "id": 1,
10       "title": "Track Maintenance #1",
11       "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
12       "notes": []
13     },
14     {
15       "id": 2,
16       "title": "Power Disruption #2",
```

Figure 33. Postman naive endpoint at page size 50 returns 50 records and 51 SQL queries.

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- GET http://127.0.0.1:8762/api/performance/incidents-o...
- GET http://127.0.0.1:8762/api/performance/incidents-n...
- GET http://127.0.0.1:8762/api/performance/incidents-o...
- GET http://127.0.0.1:8762/api/performance/incidents-n...
- GET http://127.0.0.1:8762/api/incidents/5001
- DEL http://127.0.0.1:8762/api/incidents/5001
- PUT http://127.0.0.1:8762/api/incidents/5001
- POST http://127.0.0.1:8762/api/incidents
- GET http://127.0.0.1:8762/api/incidents/1
- GET http://127.0.0.1:8762/api/incidents
- POST http://127.0.0.1:8762/api/login

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

http://127.0.0.1:8762/api/performance/incidents-optimized?page_size=50

GET http://127.0.0.1:8762/api/performance/incidents-optimized?page_size=50

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ page_size	50	
Key	Value	

Body Cookies (1) Headers (6) Test Results Status: 200 OK Time: 35 ms Size: 814 KB Save Response

Pretty Raw Preview Visualize JSON

```
1
2  {
3    "version": "optimized",
4    "page_size": 50,
5    "sql_queries": 1,
6    "retrieval_latency_ms": 17.483,
7    "records": [
8      {
9        "id": 1,
10       "title": "Track Maintenance #1",
11       "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
12       "notes": []
13     },
14     {
15       "id": 2,
16       "title": "Power Disruption #2",
```

Figure 34. Postman optimized endpoint at page size 50 returns 50 records and 1 SQL query.

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

GET http://127.0.0.1:8762/api/performance/incidents-naive?page_size=200

GET http://127.0.0.1:8762/api/performance/incident...
GET http://127.0.0.1:8762/api/performance/incidents-o...
GET http://127.0.0.1:8762/api/performance/incidents-n...
GET http://127.0.0.1:8762/api/performance/incidents-o...
GET http://127.0.0.1:8762/api/performance/incidents-n...
GET http://127.0.0.1:8762/api/incidents/5001
DEL http://127.0.0.1:8762/api/incidents/5001
PUT http://127.0.0.1:8762/api/incidents/5001
POST http://127.0.0.1:8762/api/incidents
GET http://127.0.0.1:8762/api/incidents/1
GET http://127.0.0.1:8762/api/incidents
POST http://127.0.0.1:8762/api/login

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

http://127.0.0.1:8762/api/performance/incidents-naive?page_size=200

GET http://127.0.0.1:8762/api/performance/incidents-naive?page_size=200

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ page_size	200	
Key	Value	

Body Cookies (1) Headers (6) Test Results Status: 200 OK Time: 365 ms Size: 3178 KB Save Response

Pretty Raw Preview Visualize JSON

```
1
2  {
3    "version": "naive",
4    "page_size": 200,
5    "sql_queries": 201,
6    "retrieval_latency_ms": 340.29,
7    "records": [
8      {
9        "id": 1,
10       "title": "Track Maintenance #1",
11       "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
12       "notes": []
13     },
14     {
15       "id": 2,
16       "title": "Power Disruption #2",
```

Figure 35. Postman naive endpoint at page size 200 returns 200 records and 201 SQL queries.

The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8762/api/performance/incidents-optimized?page_size=200`. The request has a query parameter `page_size=200`. The response status is 200 OK, with a time of 48 ms and a size of 31.78 KB. The response body is a JSON object with the following structure:

```

{
  "version": "optimized",
  "page_size": 200,
  "sql_queries": 1,
  "retrieval_latency_ms": 23.887,
  "records": [
    {
      "id": 1,
      "title": "Track Maintenance #1",
      "description": "Track Maintenance near Campbell caused reduced service for municipal transit passengers.",
      "notes": []
    },
    {
      "id": 2,
      "title": "Power Disruption #2",
      "description": "Power Disruption near Diridon Station caused platform congestion for municipal transit passengers.",
      "notes": []
    }
  ]
}

```

The interface also shows a history of requests on the left and a console at the bottom.

Figure 36. Postman optimized endpoint at page size 200 returns 200 records and 1 SQL query.

4.3 Measurement method and results

```

code > hw04 > benchmark_n_plus_one.py > ...
--
12  BASE_URL = "http://localhost:8762"
13  VERIFY_SEED = 262562
14  PAGE_SIZES = [10, 50, 200]
15  RUNS_PER_CONFIGURATION = 30
16
17  ROOT = Path(__file__).resolve().parents[2]
18  RAW_DIRECTORY = ROOT / "reports" / "hw04" / "raw"
19  RAW_DIRECTORY.mkdir(parents=True, exist_ok=True)
20
21  RAW_CSV = RAW_DIRECTORY / "n_plus_one_requests.csv"
22  SUMMARY_JSON = RAW_DIRECTORY / "n_plus_one_summary.json"
23  METRICS_MD = ROOT / "reports" / "hw04" / "METRICS.md"
24
25
26  def percentile(values, percentage):
27      ordered = sorted(values)
28
29      if len(ordered) == 1:
30          return ordered[0]
31
32      position = (len(ordered) - 1) * percentage
33      lower = int(position)
34      upper = min(lower + 1, len(ordered) - 1)
35      fraction = position - lower
36
37      return (
38          ordered[lower]
39          + (ordered[upper] - ordered[lower]) * fraction
40      )
41
42
43  def main():
44      session = requests.Session()
45
46      login_response = session.post(
47          f"{BASE_URL}/api/login",
48          json={
49              "email": "student@example.com",
50              "password": "Transit2562!",
51          },
52          timeout=30,
53      )
54      login_response.raise_for_status()
55
56      configurations = []
57
58      for version in ["naive", "optimized"]:
59          for page_size in PAGE_SIZES:
60              for run_number in range(
61                  1,
62                  RUNS_PER_CONFIGURATION + 1,
63              ):
64                  configurations.append(
65                      {
66                          "version": version,
67                          "page_size": page_size,
68                          "run_number": run_number,
69                      }
70              )
71
72      random.Random(VERIFY_SEED).shuffle(configurations)
73
74      results = []
75
76      for request_number, configuration in enumerate(
77          configurations,
78          start=1,

```

Figure 37. Benchmark loop runs 30 requests for each version and page size using VERIFY_SEED 262562.

```

Windows PowerShell
155/180 optimized page_size= 10 queries= 1 latency=14.889 ms
156/180 naive page_size= 50 queries= 51 latency=68.153 ms
157/180 naive page_size= 50 queries= 51 latency=65.025 ms
158/180 optimized page_size= 50 queries= 1 latency=14.909 ms
159/180 optimized page_size= 10 queries= 1 latency=11.893 ms
160/180 naive page_size= 50 queries= 51 latency=72.538 ms
161/180 optimized page_size=200 queries= 1 latency=22.262 ms
162/180 naive page_size= 50 queries= 51 latency=64.133 ms
163/180 naive page_size= 10 queries= 11 latency=23.799 ms
164/180 naive page_size= 50 queries= 51 latency=68.819 ms
165/180 naive page_size= 10 queries= 11 latency=22.770 ms
166/180 optimized page_size=200 queries= 1 latency=30.540 ms
167/180 optimized page_size= 50 queries= 1 latency=16.540 ms
168/180 optimized page_size= 10 queries= 1 latency=12.750 ms
169/180 naive page_size=200 queries=201 latency=246.178 ms
170/180 optimized page_size=200 queries= 1 latency=22.434 ms
171/180 optimized page_size= 50 queries= 1 latency=18.326 ms
172/180 naive page_size=200 queries=201 latency=234.135 ms
173/180 optimized page_size=200 queries= 1 latency=24.839 ms
174/180 optimized page_size= 10 queries= 1 latency=14.057 ms
175/180 naive page_size= 50 queries= 51 latency=73.768 ms
176/180 optimized page_size=200 queries= 1 latency=22.301 ms
177/180 optimized page_size= 10 queries= 1 latency=12.305 ms
178/180 naive page_size= 10 queries= 11 latency=26.992 ms
179/180 naive page_size= 50 queries= 51 latency=65.879 ms
180/180 optimized page_size=200 queries= 1 latency=22.094 ms

Benchmark complete.
Total requests: 180
Raw results: D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw04\raw\n_plus_one_requests.csv
Summary: D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw04\raw\n_plus_one_summary.json
Metrics: D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw04\METRICS.md

Summary:
{'version': 'naive', 'page_size': 10, 'request_count': 30, 'sql_queries_per_request': 11, 'p50_ms': 23.96, 'p95_ms': 32.835, 'p99_ms': 36.404}
{'version': 'naive', 'page_size': 50, 'request_count': 30, 'sql_queries_per_request': 51, 'p50_ms': 68.424, 'p95_ms': 98.369, 'p99_ms': 111.576}
{'version': 'naive', 'page_size': 200, 'request_count': 30, 'sql_queries_per_request': 201, 'p50_ms': 230.707, 'p95_ms': 322.926, 'p99_ms': 396.742}
{'version': 'optimized', 'page_size': 10, 'request_count': 30, 'sql_queries_per_request': 1, 'p50_ms': 13.28, 'p95_ms': 16.97, 'p99_ms': 18.667, 'p50_speedup': 1.8}
{'version': 'optimized', 'page_size': 50, 'request_count': 30, 'sql_queries_per_request': 1, 'p50_ms': 16.561, 'p95_ms': 23.029, 'p99_ms': 27.154, 'p50_speedup': 4.13}
{'version': 'optimized', 'page_size': 200, 'request_count': 30, 'sql_queries_per_request': 1, 'p50_ms': 22.407, 'p95_ms': 32.206, 'p99_ms': 54.808, 'p50_speedup': 10.3}

(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562> $rows = Import-Csv .\reports\hw04\raw\n_plus_one_requests.csv
>> $rows.Count
180
>> $rows | Group-Object version, page_size |
>> Select-Object Name, Count |
>> Format-Table -AutoSize
Name Count
----
naive, 10 30
optimized, 50 30
naive, 200 30
optimized, 10 30
optimized, 200 30
naive, 50 30

```

Figure 38. Completed 180-request run and six groups of 30 measurements.

Page size	Version	SQL stmts/req	p50 ms	p95 ms	p99 ms	p50 speedup
10	Naive	11	23.960	32.835	36.404	-
10	Fixed	1	13.280	16.970	18.667	1.80x
50	Naive	51	68.424	98.369	111.576	-
50	Fixed	1	16.561	23.029	27.154	4.13x
200	Naive	201	230.707	322.926	396.742	-
200	Fixed	1	22.407	32.206	54.808	10.30x

Observation

The optimized endpoint is faster at every page size, and the advantage grows from 1.80x at 10 records to 10.30x at 200 records. The naive version executes N additional note queries, so network, parsing, and database round-trip overhead rises with the page size. The optimized version holds the query count at one; its latency grows mainly because more joined rows must be transferred and grouped in Python.

4.4 Index and EXPLAIN comparison

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Query 1

```

1 • USE s2562_rel;
2
3 EXPLAIN
4 SELECT
5     i.id,
6     i.title,
7     i.description,
8     n.id AS note_id,
9     n.note
10  FROM (
11      SELECT id
12      FROM incidents
13      ORDER BY id
14      LIMIT 200
15  ) AS page
16  JOIN incidents AS i
17    ON i.id = page.id
18  LEFT JOIN incident_notes AS n

```

Result Grid

Filter Rows: Export: Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	200	100.00	Using temporary; Using filesort
1	PRIMARY	i	NULL	eq_ref	PRIMARY	PRIMARY	4	page.id	1	100.00	NULL
1	PRIMARY	n	NULL	ALL	NULL	NULL	NULL	NULL	200	100.00	Using where; Using join buffer (hash join)
2	DERIVED	incidents	NULL	index	NULL	PRIMARY	4	NULL	200	100.00	Using index

Result 17 x

Read Only

Figure 39. EXPLAIN before indexing incident_notes.incident_id shows a full scan of the related table.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL:

```

1 USE s2562_rel;
2
3 EXPLAIN
4 SELECT
5   i.id,
6   i.title,
7   i.description,
8   n.id AS note_id,
9   n.note
10  FROM (
11    SELECT id
12    FROM incidents
13    ORDER BY id
14    LIMIT 200
15  ) AS page
16  JOIN incidents AS i
17    ON i.id = page.id
18  LEFT JOIN incident_notes AS n

```

The Result Grid shows the following information:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	200	100.00	Using temporary; Using filesort
1	PRIMARY	i	NULL	eq_ref	PRIMARY	PRIMARY	4	page.id	1	100.00	Using index
1	PRIMARY	n	NULL	ref	idx_incident_notes_incident_id	idx_incident_notes_incident_id	4	page.id	1	100.00	Using index
2	DERIVED	incidents	NULL	index	PRIMARY	PRIMARY	4	NULL	200	100.00	Using index

Figure 40. EXPLAIN after adding `idx_incident_notes_incident_id` shows indexed note lookup.

Before the index, MySQL reports an ALL access type for incident_notes and estimates scanning the related table. After the index, the plan uses the incident_id index and a ref lookup. The join can locate notes associated with each incident without scanning every note row.

5 Part 4 Grounded RAG Question Answering System

5.1 Corpus, chunking, embeddings, and retrieval

```

code > hw04 > rag.py > ...
19  MODEL_NAME = "qwen3:1.7b"
20  EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
21  CHUNK_SIZE = 500
22  CHUNK_OVERLAP = 50
23  TOP_K = 3
24  REFUSAL = (
25      "I cannot answer this question from the provided documents"
26  )
27
28
29  def load_text(path):
30      if path.suffix.lower() == ".pdf":
31          reader = PdfReader(path)
32          return "\n".join(
33              page.extract_text() or ""
34              for page in reader.pages
35          )
36
37      return path.read_text(
38          encoding="utf-8",
39          errors="ignore"
40      )
41
42
43  def create_chunks(text, source):
44      text = re.sub(r"\s+", " ", text).strip()
45      chunks = []
46      start = 0
47      number = 1
48
49      while start < len(text):
50          end = min(start + CHUNK_SIZE, len(text))
51          chunk_text = text[start:end].strip()
52
53          if chunk_text:
54              chunks.append(
55                  {

```

Figure 41. RAG configuration uses chunk size 500, overlap 50, top-k 3, and the exact refusal sentence.

```

code > hw04 > rag.py > build_index
90
91 def build_index(chunks, embedding_model):
92     texts = [chunk["text"] for chunk in chunks]
93
94     embeddings = embedding_model.encode(
95         texts,
96         convert_to_numpy=True,
97         normalize_embeddings=True,
98         show_progress_bar=True
99     ).astype("float32")
100
101     index = faiss.IndexFlatIP(embeddings.shape[1])
102     index.add(embeddings)
103
104     return index
105
106
107 def retrieve(
108     question,
109     k,
110     chunks,
111     index,
112     embedding_model
113 ):
114     query = embedding_model.encode(
115         [question],
116         convert_to_numpy=True,
117         normalize_embeddings=True
118     ).astype("float32")
119
120     scores, indices = index.search(query, k)
121     results = []
122
123     for rank, (score, index_number) in enumerate(
124         zip(scores[0], indices[0]),
125         start=1
126     ):

```

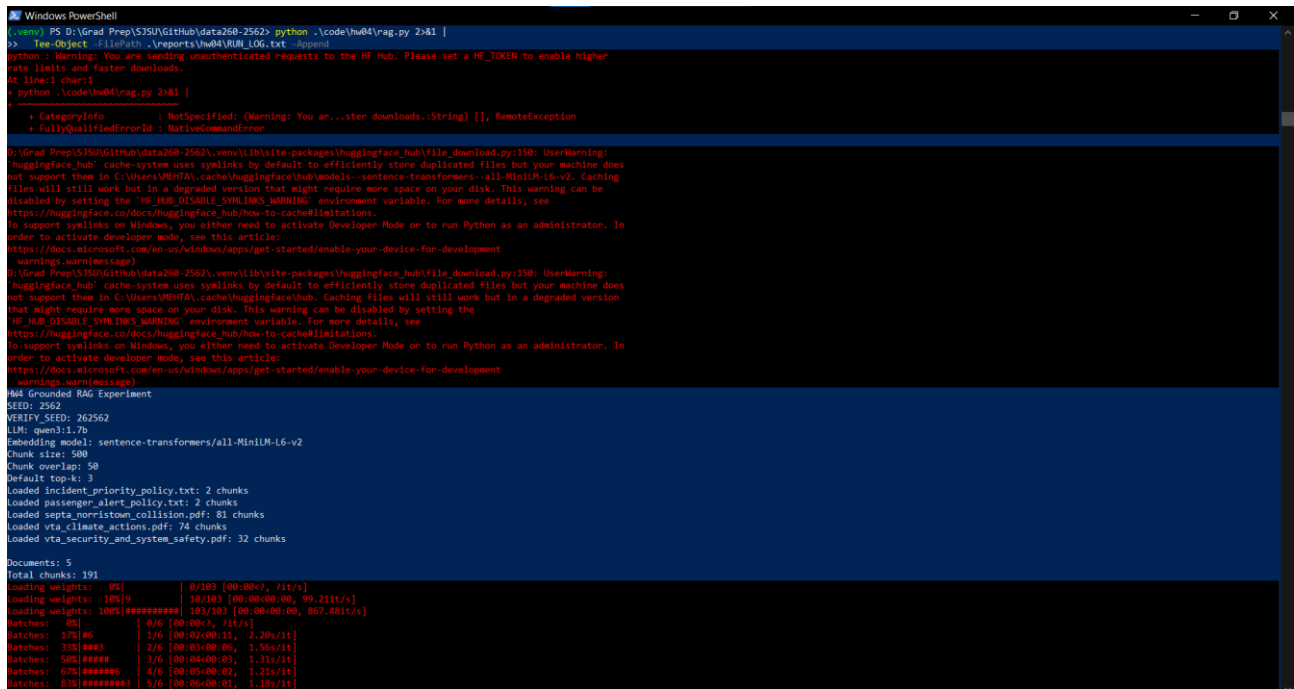
Figure 42. MiniLM embeddings are normalized and loaded into a FAISS inner-product index.

```

code > hw04 > rag.py > retrieve
107 def retrieve(
108     question,
109     k,
110     chunks,
111     index,
112     embedding_model
113 ):
114     query = embedding_model.encode(
115         [question],
116         convert_to_numpy=True,
117         normalize_embeddings=True
118     ).astype("float32")
119
120     scores, indices = index.search(query, k)
121     results = []
122
123     for rank, (score, index_number) in enumerate(
124         zip(scores[0], indices[0]),
125         start=1
126     ):
127         chunk = chunks[int(index_number)]
128
129         results.append(
130             {
131                 "rank": rank,
132                 "source": chunk["source"],
133                 "chunk_id": chunk["chunk_id"],
134                 "score": round(float(score), 6),
135                 "text": chunk["text"]
136             }
137         )
138
139     return results
140
141
142 def print_retrieval(question_id, results):
143     print(f"\n--- {question_id}: retrieved chunks ---")
144
145     for result in results:
146         print(
147             f"Rank {result['rank']} | "
148             f"Source: {result['source']} | "
149             f"Chunk: {result['chunk_id']} | "
150             f"Score: {result['score']}"
151         )
152         print(result["text"])
153         print()
154
155

```

Figure 43. Retrieval returns ranked chunks with source, chunk ID, score, and text before generation.



```

Windows PowerShell
PS D:\Grad Prep\535\Github\data260-2562> python .\code\hw04\rag.py 2>&1 |
tee -o object -file path .\reports\hw04\RUN_LOG.txt Append
python : Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
rate limits and faster downloads.
Hit line 1 char 1
python .\code\hw04\rag.py 2>&1 |
+ CategoryInfo          : NotSpecified; (Warning: You are...ster downloads.:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

D:\Grad Prep\535\Github\data260-2562> python .\code\hw04\rag.py 2>&1 | tee -o object -file path .\reports\hw04\RUN_LOG.txt Append
WARNING: huggingface_hub cache-system uses symlinks by default to efficiently store duplicated files but your machine does
not support them in C:\Users\VM01A1\cache\huggingface\hub\models--sentence-transformers--all-MiniLM-L6-v2. Caching
files will still work but in a degraded version that might require more space on your disk. This warning can be
disabled by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING' environment variable. For more details, see
https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In
order to activate developer mode, see this article:
https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
warnings.warn(message)
D:\Grad Prep\535\Github\data260-2562> python .\code\hw04\rag.py 2>&1 | tee -o object -file path .\reports\hw04\RUN_LOG.txt Append
WARNING: huggingface_hub cache-system uses symlinks by default to efficiently store duplicated files but your machine does
not support them in C:\Users\VM01A1\cache\huggingface\hub. Caching files will still work but in a degraded version
that might require more space on your disk. This warning can be disabled by setting the
HF_HUB_DISABLE_SYMLINKS_WARNING environment variable. For more details, see
https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In
order to activate developer mode, see this article:
https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
warnings.warn(message)

H4d Grounded RAG Experiment
SEED: 2562
VERIFY SEED: 262562
LLM: qwen3:1.7b
Embedding model: sentence-transformers/all-MiniLM-L6-v2
Chunk size: 500
Chunk overlap: 50
Default top-k: 3
Loaded incident_priority_policy.txt: 2 chunks
Loaded passenger_alert_policy.txt: 2 chunks
Loaded septa_norristown_collision.pdf: 81 chunks
Loaded vta_climate_actions.pdf: 74 chunks
Loaded vta_security_and_system_safety.pdf: 32 chunks

Documents: 5
Total chunks: 191
Loading weights: 0% | 0/103 [00:00<, 71t/s]
Loading weights: 10%|9 | 10/103 [00:00:00.00, 99.211t/s]
Loading weights: 100|#####| 103/103 [00:00:00.00, 867.481t/s]
Batches: 0% | 0/6 [00:00<, 71t/s]
Batches: 17%|40 | 1/6 [00:02:00:11, 2.20s/1t]
Batches: 33%|80 | 2/6 [00:03:00:06, 1.56s/1t]
Batches: 50%|120 | 3/6 [00:04:00:03, 1.31s/1t]
Batches: 67%|160 | 4/6 [00:05:00:02, 1.22s/1t]
Batches: 83%|200 | 5/6 [00:06:00:01, 1.10s/1t]

```

Figure 44. RAG run loads five corpus files, creates the FAISS index, and begins retrieval.

The five-document corpus contains incident_priority_policy.txt, passenger_alert_policy.txt, septa_norristown_collision.pdf, vta_climate_actions.pdf, and vta_security_and_system_safety.pdf. Each chunk retains its source filename and chunk identifier.

5.2 Context engineering and three configurations

```

code > hw04 > rag.py > context_engineer
156 def context_engineer(results):
157     selected = []
158     seen_text = set()
159
160     for result in results:
161         normalized = re.sub(
162             r"\W+",
163             " ",
164             result["text"].lower()
165         ).strip()
166
167         signature = normalized[:250]
168
169         if result["score"] < 0.30:
170             continue
171
172         if signature in seen_text:
173             continue
174
175         seen_text.add(signature)
176         selected.append(result)
177
178     return selected[:3]
179
180
181 def build_context(results):
182     sections = []
183
184     for number, result in enumerate(results, start=1):
185         sections.append(
186             f"[Source {number}: {result['source']}; "
187             f"chunk_id={result['chunk_id']}] \n"
188             f"{result['text']}"
189         )
190
191     return "\n\n".join(sections)
192
193
194 def call_ollama(prompt):
195     payload = {
196         "model": MODEL_NAME,
197         "messages": [
198             {
199                 "role": "user",
200                 "content": prompt
201             }
202         ],
203         "stream": False,
204         "think": False,
205         "options": {
206             "temperature": 0.0,
207             "num_ctx": 4096,
208             "num_predict": 180,
209             "seed": 2562
210         }

```

Figure 45. Context engineering removes low-scoring and duplicate chunks, labels sources, and limits the selected context.

```

code > hw04 > rag.py > make_prompt
---
231 def make_prompt(configuration, question, results):
232     if configuration == "A":
233         return (
234             "Answer the following question concisely.\n\n"
235             f"Question: {question}"
236         )
237
238     context = build_context(results)
239
240     if configuration == "B":
241         return (
242             "Use the following retrieved text to answer the
243             question.\n\n"
244             f"{context}\n\n"
245             f"Question: {question}"
246         )
247
248     return (
249         "Answer only from the provided context.\n"
250         "Cite supporting evidence using [Source 1], "
251         "[Source 2], and so on.\n"
252         "Do not use outside knowledge.\n"
253         "If the evidence is insufficient, respond exactly:
254         f"{REFUSAL}\n\n"
255         f"Context:\n{context}\n\n"
256         f"Question: {question}"
257     )
258
259
260 def expected_keywords(question_id):
261     mapping = {
262         "Q1": ["five minutes"],
263         "Q2": ["p1", "15 minutes"],
264         "Q3": [],
265         "Q4": [],
266         "Q5": [],
267         "Q6": []
268     }
269
270     return mapping[question_id]
271
272
273 def evaluate(question, configuration, answer, retrieval):
274     expected_sources = set(question["expected_sources"])
275     retrieved_sources = {
276         result["source"]
277         for result in retrieval
278     }
279
280     refusal_needed = question["id"] in {"Q5", "Q6"}
281     refused = REFUSAL.lower() in answer.lower()
282
283     keywords = expected_keywords(question["id"])
284
285     if refusal_needed:

```

Figure 46. Prompt construction implements No RAG, Basic RAG, and grounded Context RAG.

```

code > hw04 > rag.py > main > configurations
353 def main():
...
404 configurations = {
405     "A": [],
406     "B": retrieval,
407     "C": context_engineer(retrieval)
408 }
409
410 for configuration, context_results in (
411     configurations.items()
412 ):
413     prompt = make_prompt(
414         configuration,
415         question["question"],
416         context_results
417     )
418
419     started = time.perf_counter()
420     answer = call_ollama(prompt)
421     latency_ms = round(
422         (time.perf_counter() - started) * 1000,
423         3
424     )
425
426     print(
427         f"\n{question['id']} "
428         f"Configuration {configuration}"
429     )
430     print(answer)
431
432     comparison_output.append(
433         {
434             "question_id": question["id"],
435             "question_type": question["type"],
436             "configuration": configuration,
437             "answer": answer,
438             "latency_ms": latency_ms,
439             "retrieved_sources": "; ".join(
440                 result["source"]
441                 for result in context_results
442             )
443         }
444     )
445

```

Figure 47. Three-configuration experiment loop executes and records A, B, and C for every question.

```

code > hw04 > rag.py > call_ollama
---
194 def call_ollama(prompt):
195     payload = {
196         "model": MODEL_NAME,
197         "messages": [
198             {
199                 "role": "user",
200                 "content": prompt
201             }
202         ],
203         "stream": False,
204         "think": False,
205         "options": {
206             "temperature": 0.0,
207             "num_ctx": 4096,
208             "num_predict": 180,
209             "seed": 2562
210         }
211     }
212
213     request = urllib.request.Request(
214         "http://127.0.0.1:11434/api/chat",
215         data=json.dumps(payload).encode("utf-8"),
216         headers={"Content-Type": "application/json"},
217         method="POST"
218     )
219
220     with urllib.request.urlopen(
221         request,
222         timeout=600
223     ) as response:
224         data = json.loads(
225             response.read().decode("utf-8")
226         )
227
228     return data["message"]["content"].strip()
229
230
231 def make_prompt(configuration, question, results):
232     if configuration == "A":
233         return (
234             "Answer the following question concisely.\n\n"
235             f"Question: {question}"
236         )
237
238     context = build_context(results)
239
240     if configuration == "B":
241         return (
242             "Use the following retrieved text to answer the "
243             "question.\n\n"
244             f"{context}\n\n"
245             f"Question: {question}"
246         )
247
248     return (

```

Figure 48. Ollama call uses qwen3:1.7b with temperature 0.0, context 4096, and SEED 2562.

5.3 Six-question test set

```

data > hw04 > {} questions.json > ...
1  [
2    {
3      "id": "Q1",
4      "type": "one_chunk",
5      "question": "How quickly must the first passenger alert be issued for a P1 incident?",
6      "expected_sources": ["passenger_alert_policy.txt"],
7      "expected_answer": "Within five minutes."
8    },
9    {
10     "id": "Q2",
11     "type": "two_chunks",
12     "question": "How should a signal failure with an immediate safety threat be classified, and how frequently should passengers receive updates?",
13     "expected_sources": [
14       "incident_priority_policy.txt",
15       "passenger_alert_policy.txt"
16     ],
17     "expected_answer": "It is a P1 incident requiring immediate escalation, with passenger updates every 15 minutes."
18   },
19   {
20     "id": "Q3",
21     "type": "similar_information",
22     "question": "What safety-preparation practices are described across the transit safety documents?",
23     "expected_sources": [
24       "vta_security_and_system_safety.pdf",
25       "septa_norristown_collision.pdf"
26     ],
27     "expected_answer": "Use the retrieved evidence to summarize related safety-preparation practices."
28   },
29   {
30     "id": "Q4",
31     "type": "ambiguous",
32     "question": "What response is required for a transit incident?",
33     "expected_sources": [
34       "incident_priority_policy.txt",
35       "passenger_alert_policy.txt"
36     ],
37     "expected_answer": "The required response depends on the incident priority and available details."
38   },
39   {
40     "id": "Q5",
41     "type": "not_in_documents",
42     "question": "What will the exact VTA adult fare be in January 2035?",
43     "expected_sources": [],
44     "expected_answer": "I cannot answer this question from the provided documents"
45   },
46   {
47     "id": "Q6",
48     "type": "unrelated",
49     "question": "Who won the 2026 World Series?",
50     "expected_sources": [],
51     "expected_answer": "I cannot answer this question from the provided documents"
52   }
53 ]

```

Figure 49. Precommitted six-question set with expected sources and answers.

```

Windows PowerShell

--- Q1: retrieved chunks ---
Rank 1 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-1 | Score: 0.682065
Municipal Transit Passenger Alert Policy For Priority 1 incidents, the first public passenger alert must be issued within five minutes after operations control confirms the incident. Updates must be published at least every fifteen minutes while the disruption remains active. Alerts should identify the affected route, location, expected delay, available alternatives, and the time of the next update. Approved communication channels include station displays, the transit website, mobile notifications, and official social-media accounts. Priority 2 incidents require a public alert when the expected disruption exceeds ten minutes. Priority 3 incidents may be included in scheduled service notices.
Rank 2 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-2 | Score: 0.513327
Rank 3 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-72 | Score: 0.450489
AR-15-01: Washington, DC: HTSB. (b) This recommendation is currently classified Open/Acceptable Response. See CAROL for this recommendation's complete history. Railroad Investigation Report RIR-26-05 17 personnel to train emergency responders on transit emergencies and added estimated times of arrival for SEPTA personnel to the information it provides emergency responders. Both changes are intended to support quickly and effectively securing an accident scene. 3 Probable Cause The National Transportation Safety Board

Q1 Configuration A
The first passenger alert must be issued immediately for a P1 incident.

Q1 Configuration B
For a Priority 1 incident, the first public passenger alert must be issued within **five minutes** after operations control confirms the incident.

Q1 Configuration C
The first passenger alert for a P1 incident must be issued within five minutes after operations control confirms the incident. This is stated in [Source 1].

```

Figure 50. Q1 retrieval and A/B/C answers; grounded configurations return the five-minute requirement.

```

Windows PowerShell

--- Q2: retrieved chunks ---
Rank 1 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-1 | Score: 0.610524
Municipal Transit Passenger Alert Policy For Priority 1 Incidents, the first public passenger alert must be issued within five minutes after operations control confirms the incident. Updates must be published at least every fifteen minutes while the disruption remains active. Alerts should identify the affected route, location, expected delay, available alternatives, and the time of the next update. Approved communication channels include station displays, the transit website, mobile notification.

Rank 2 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-72 | Score: 0.531376
18-15-01. Washington, DC: HHSB. (b) This recommendation is currently classified OpenAcceptable Response. See CAROL for this recommendation's complete history. Railroad Investigation Report RIR-26-05 17 personnel to train emergency responders on transit emergencies and added estimated times of arrival for SEPTA personnel to the information it provides emergency responders. Both changes are intended to support quickly and effectively securing an accident scene. 3 Probable Cause The National Transportation Safety Board (NTSB) has determined that the collision was caused by a signal failure. The signal system was unable to detect the imminent collision and intervene. 4 Lessons Learned This collision could have been prevented by a train control system that did not depend solely on operator vigilance. A system that leaves the operator as a single point of failure is vulnerable to safety risks such as operator fatigue, distraction, or incapacitation. Requiring TBTC on rail transit systems,

Q2 Configuration A
A signal failure with an immediate safety threat should be classified as a **critical alert** and passengers should receive updates **immediately**.

Q2 Configuration B
Based on the retrieved text:

1. **Classification of a signal failure with an immediate safety threat**:
The text from **Source 2** (septa_norristown_collision.pdf) indicates that such a situation is **classified as OpenAcceptable Response**. This classification suggests that the issue is considered acceptable and requires a response, but it does not specify a particular classification like "Critical" or "Emergency" in the text. However, the context implies that a signal failure with an immediate safety threat is a **high-priority incident** that requires immediate action.

2. **Frequency of passenger updates**:
The text from **Source 1** (passenger_alert_policy.txt) states that **updates must be published at least every fifteen minutes** while the disruption remains active. This frequency applies to **Priority 1 incidents**, which are those with an immediate safety threat.

Q2 Configuration C
According to the provided context, a signal failure with an immediate safety threat should be classified as an "OpenAcceptable Response" [Source 2]. Passengers should receive updates at least every fifteen minutes while the disruption remains active [Source 1].

```

Figure 51. Q2 retrieval misses the incident-priority source at $k=3$ and produces an unsuitable classification.

```

Windows PowerShell

--- Q3: retrieved chunks ---
Rank 1 | Source: vta_security_and_system_safety.pdf | Chunk: vta_security_and_system_safety.pdf-chunk-3 | Score: 0.551316
Improper passenger drop-off and loading. Fire and life safety considerations involve preventive design criteria and those that provide protection for people and property in the event an emergency should occur. 4.11.1 VTA AND OTHER TRANSIT FACILITIES Security and safety measures are already in place to serve current bus transit operations and related pedestrian activities near existing transit facilities and bus stops in the SVRITC. VTA's Protective Services provides security for VTA bus and light rail.

Rank 2 | Source: vta_security_and_system_safety.pdf | Chunk: vta_security_and_system_safety.pdf-chunk-12 | Score: 0.549289
In subways that are inaccessible to the public. Wherever the alignment is at grade, warning signs are posted on the security fences adjacent to the tracks to prevent trespassing and to warn of the dangers of entering the track area containing the electrified third rail. The BART Police Department is also responsible for participating in design review of new facilities to ensure consistency with security and safety standards. For example, public areas and parking lots are well illuminated and designed to ensure safety and prevent accidents.

Rank 3 | Source: vta_security_and_system_safety.pdf | Chunk: vta_security_and_system_safety.pdf-chunk-1 | Score: 0.546792
Silicon Valley Rapid Transit Corridor Final EIS Affected Environment Security and System Safety 4.11-1 4.11 SECURITY AND SYSTEM SAFETY Security refers to the prevention of unlawful acts resulting in harm to persons or damage to property. In a broader sense, it also implies freedom from threats or uncertainty about the likelihood of threatening acts. Crime and antisocial behavior are potential problems in any public environment. System Safety refers to the prevention of accidents to the riding public.

Q3 Configuration A
The transit safety documents describe practices such as regular equipment inspections, emergency response plans, driver training, and compliance with safety regulations.

Q3 Configuration B
The safety-preparation practices described across the transit safety documents include:

1. **Security and Safety Measures**: The documents emphasize the implementation of security and safety measures to prevent unlawful acts, protect persons and property, and ensure the prevention of accidents in transit facilities.

2. **Preventive Design Criteria**: The documents mention the use of preventive design criteria to address potential safety issues, such as improper passenger drop-off and loading, and to ensure that transit facilities are safe and secure.

3. **Fire and Life Safety Considerations**: These documents highlight the importance of fire and life safety considerations, including the installation of warning signs, illumination of public areas, and the design of security fences to prevent trespassing and ensure safety in track areas.

4. **Collaboration with Law Enforcement**: The documents note the involvement of law enforcement agencies, such as the BART Police Department, in the design review of new facilities.

Q3 Configuration C
The safety-preparation practices described across the transit safety documents include:

1. **Security and Safety Measures**: The documents mention the implementation of security and safety measures for transit facilities, such as the VTA's Protective Services providing security for bus and light rail facilities. These measures include preventive design criteria and protection for people and property in the event of an emergency.

2. **Fire and Life Safety Considerations**: The documents discuss fire and life safety considerations, including the prevention of improper passenger drop-off and loading, and the installation of warning signs to prevent trespassing and warn of dangers in track areas.

3. **Design Review and Compliance**: The BART Police Department is involved in the design review of new facilities to ensure compliance with security and safety standards.

4. **Public Areas and Parking Lots**: The documents note that public areas and parking lots are well illuminated and designed to ensure safety and prevent accidents.

```

Figure 52. Q3 retrieves VTA safety evidence and compares all three configurations.

```

Windows PowerShell

--- Q4: retrieved chunks ---
Rank 1 | Source: incident_priority_policy.txt | Chunk: incident_priority_policy.txt-chunk-1 | Score: 0.611442
Municipal Transit Incident Priority Policy Priority 1 incidents include complete signal-system failure, collision, fire, evacuation, or any condition presenting an immediate threat to passenger safety. Priority 1 incidents must be escalated immediately to the operations control center and emergency responders. Priority 2 incidents include major delays, partial route suspension, station closure, and equipment failures that significantly affect service but do not create an immediate safety threat.

Rank 2 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-2 | Score: 0.582024
Displays, the transit website, mobile notifications, and official social-media accounts. Priority 2 incidents require a public alert when the expected disruption exceeds ten minutes. Priority 3 incidents may be included in scheduled service notices.

Rank 3 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-72 | Score: 0.557942
M-15-01: Washington, DC: HHSB. (0) This recommendation is currently classified Open/Acceptable Response. See CAROL for this recommendation's complete history. Railroad Investigation Report RIR-26-05 17 personnel to train emergency responders on transit emergencies and added estimated times of arrival for SEPTA personnel to the information it provides emergency responders. Both changes are intended to support quickly and effectively securing an accident scene. 3 Probably Cause The National Transit

Q4 Configuration A
A transit incident requires a coordinated response, including immediate assessment, evacuation, and deployment of emergency services.

Q4 Configuration B
The response required for a transit incident depends on its priority level:

- **Priority 1 incidents** (e.g., complete signal-system failure, collision, fire, evacuation, or any condition presenting an immediate threat to passenger safety) must be **escalated immediately to the operations control center and emergency responders**.

- **Priority 2 incidents** (e.g., major delays, partial route suspension, station closure, and equipment failures that significantly affect service but do not create an immediate safety threat) require a **public alert** when the expected disruption exceeds ten minutes.

- **Priority 3 incidents** may be included in **scheduled service notices**.

Thus, the required response depends on the specific priority of the incident. Priority 1 incidents require immediate escalation and emergency response, while Priority 2 incidents require public alerts if the disruption exceeds ten minutes.

Q4 Configuration C
The response required for a transit incident depends on its priority level.

- **Priority 1 incidents** must be escalated immediately to the operations control center and emergency responders. [Source 1]
- **Priority 2 incidents** require a public alert when the expected disruption exceeds ten minutes. [Source 2]
- **Priority 3 incidents** may be included in scheduled service notices. [Source 3]

I cannot answer this question from the provided documents

```

Figure 53. Q4 demonstrates ambiguity and priority-dependent response behavior.

```

Windows PowerShell

--- Q5: retrieved chunks ---
Rank 1 | Source: vta_climate_actions.pdf | Chunk: vta_climate_actions.pdf-chunk-46 | Score: 0.412286
Clara Valley Water Agency) to reduce the overall vulnerability of VIA's transportation system to the effects of permanent inundation and coastal flooding. Support community participation in these efforts. Engineering & Program Delivery V
Variable ADIF-3.9: Incorporate future sea-level rise, permanent inundation, and precipitation projections into long-term infrastructure planning processes, influencing decisions on expansion, relocation, or retrofitting of assets. For exa
ple, many of VIA's facil

Rank 2 | Source: vta_climate_actions.pdf | Chunk: vta_climate_actions.pdf-chunk-10 | Score: 0.350567
der community about adaptation-related needs that would be mutually beneficial for all parties that use or otherwise rely on VIA's transportation network. External Affairs Mid-Term AD-CC-2.4: Leverage and potentially modify existing mobi
le applications (e.g., "VTAAlerts" app), along with social media accounts, to inform the public on climate hazards (e.g., extreme heat, flooding), alternative routes, and climate resilience initiatives, and to provide real-time reports fr
om the public to help VIA

Rank 3 | Source: vta_climate_actions.pdf | Chunk: vta_climate_actions.pdf-chunk-73 | Score: 0.332413
11 contributions to VIA's operational GHG inventory. Human Resources Low Variable GHG-FE-4.2: Encourage and enable VIA employees to use transit, carpool, bike, and telecommute to work to reduce single-occupancy vehicle commute trips and
VMT. Human Resources Medium Variable Materials and Waste (WM) GHG-WM-1.1: Require procurement and operational practices that avoid generation of waste (e.g., reusable materials, reduced packaging, and compostable products). Finance, Budg
et, & Real Estate Low Near

Q5 Configuration A
The exact VIA adult fare in January 2035 will be determined by VIA's annual budget and fare adjustment policy, which is not publicly disclosed.

Q5 Configuration B
The exact VIA adult fare in January 2035 is not provided in the retrieved text. The text discusses climate adaptation strategies, infrastructure planning, community engagement, and sustainability initiatives related to VIA's transportati
on system. It does not include any information about fare structures or financial details for future dates. Therefore, the answer cannot be determined from the given sources.

Q5 Configuration C
I cannot answer this question from the provided documents

```

Figure 54. Q5 is unsupported; Context RAG returns the exact required refusal.

```

Windows PowerShell

--- Q6: retrieved chunks ---
Rank 1 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-81 | Score: 0.133348
al Transportation Safety Board Records Management Division, CIO-40 490 L'Enfant Plaza, SW Washington, DC 20594 (800) 877-6799 or (202) 314-6551

Rank 2 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-62 | Score: 0.128221
to 5 mph but did not reconfigure its ATC to enforce the new speed. In the absence of a system to prevent operators from exceeding the new maximum authorized speed, operator performance remains a single point of failure. The rule change w
ill therefore not prevent future collisions involving operator error. SEPTA has installed trip stops at Norristown Station ahead of the bumping posts to automatically apply a train's air brakes before a collision. However, the available
braking distance is limite

Rank 3 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-43 | Score: 0.120805
In the emergency. At the time of the collision, SEPTA dispatchers did not have access to cameras or other sources of information about accidents other than through radio and phone calls, and Norristown Station did not always have a stati
on manager present. No station manager was present at the time of the collision. 1.8 Postaccident Actions On May 22, 2025, in response to the collision, SEPTA issued an operations bulletin imposing a speed restriction near Norristown Sta
tion, reducing the maximum

Q6 Configuration A
The 2026 World Series winner was the Los Angeles Dodgers.

Q6 Configuration B
The question is not related to the provided text. The text discusses a transportation safety incident involving SEPTA and its response to a collision, but it does not mention anything about sports, the World Series, or any athletes. Ther
efore, the answer is:

**The text does not provide information about who won the 2026 World Series.**

Q6 Configuration C
I cannot answer this question from the provided documents

```

Figure 55. Q6 is unrelated; Context RAG returns the exact required refusal.

5.4 Top k sweep

```

code > hw04 > rag.py > main > sweep_question
353 def main():
461     sweep_question = questions[1]
462     sweep_output = []
463
464     print("\n=== TOP-K SWEEP: Q2 ===")
465
466     for k in [1, 3, 5]:
467         retrieval = retrieve(
468             sweep_question["question"],
469             k,
470             chunks,
471             index,
472             embedding_model
473         )
474
475         print_retrieval(
476             f"{sweep_question['id']} k={k}",
477             retrieval
478         )
479
480         engineered = context_engineer(retrieval)
481         prompt = make_prompt(
482             "C",
483             sweep_question["question"],
484             engineered
485         )
486         answer = call_ollama(prompt)
487
488         print(f"Context-RAG answer at k={k}:")
489         print(answer)
490
491         sweep_output.append(
492             {
493                 "question_id": sweep_question["id"],
494                 "k": k,
495                 "retrieved": retrieval,
496                 "selected_after_filtering": engineered,
497                 "answer": answer
498             }
499         )
500

```

Figure 56. Top-k sweep code runs Q2 at k values 1, 3, and 5.

```

Windows PowerShell
--- TOP-K SHEEP: Q2 ---

--- Q2 k=1: retrieved chunks ---
Rank 1 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-1 | Score: 0.618524
Municipal Transit Passenger Alert Policy For Priority 1 incidents, the first public passenger alert must be issued within five minutes after operations control confirms the incident. Updates must be published at least every fifteen minutes while the disruption remains active. Alerts should identify the affected route, location, expected delay, available alternatives, and the time of the next update. Approved communication channels include station displays, the transit website, mobile notification

Context-RAG answer at k=1:
The signal failure with an immediate safety threat should be classified as a Priority 1 incident [Source 1]. Passengers should receive updates at least every fifteen minutes while the disruption remains active [Source 1].

--- Q2 k=3: retrieved chunks ---
Rank 1 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-1 | Score: 0.618524
Municipal Transit Passenger Alert Policy For Priority 1 incidents, the first public passenger alert must be issued within five minutes after operations control confirms the incident. Updates must be published at least every fifteen minutes while the disruption remains active. Alerts should identify the affected route, location, expected delay, available alternatives, and the time of the next update. Approved communication channels include station displays, the transit website, mobile notification

Rank 2 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-72 | Score: 0.531376
AR-15-01. Washington, DC: NTSB. (b) This recommendation is currently classified Open/Acceptable Response. See CAROL for this recommendation's complete history. Railroad Investigation Report RIR-26-05 17 personnel to train emergency responders on transit emergencies and added estimated times of arrival for SEPTA personnel to the information it provides emergency responders. Both changes are intended to support quickly and effectively securing an accident scene. 3 Probably Cause The National Transit

Rank 3 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-74 | Score: 0.520771
d from his duties for unknown reasons and failed to apply the vehicle's brakes, the automatic train control system was unable to detect the imminent collision and intervene. 4 Lessons Learned This collision could have been prevented by a train control system that did not depend solely on operator vigilance. A system that leaves the operator as a single point of failure is vulnerable to safety risks such as operator fatigue, distraction, or incapacitation. Requiring TBTC on rail transit systems,

Context-RAG answer at k=3:
According to the provided context, a signal failure with an immediate safety threat should be classified as an "Open/Acceptable Response" [Source 2]. Passengers should receive updates at least every fifteen minutes while the disruption remains active [Source 1].

--- Q2 k=5: retrieved chunks ---
Rank 1 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-1 | Score: 0.618524
Municipal Transit Passenger Alert Policy For Priority 1 incidents, the first public passenger alert must be issued within five minutes after operations control confirms the incident. Updates must be published at least every fifteen minutes while the disruption remains active. Alerts should identify the affected route, location, expected delay, available alternatives, and the time of the next update. Approved communication channels include station displays, the transit website, mobile notification

Rank 2 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-72 | Score: 0.531376
AR-15-01. Washington, DC: NTSB. (b) This recommendation is currently classified Open/Acceptable Response. See CAROL for this recommendation's complete history. Railroad Investigation Report RIR-26-05 17 personnel to train emergency responders on transit emergencies and added estimated times of arrival for SEPTA personnel to the information it provides emergency responders. Both changes are intended to support quickly and effectively securing an accident scene. 3 Probably Cause The National Transit

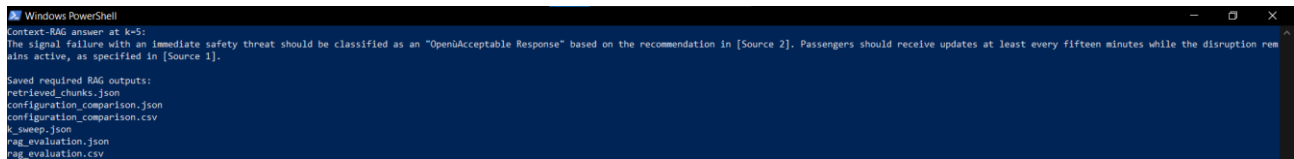
Rank 3 | Source: septa_norristown_collision.pdf | Chunk: septa_norristown_collision.pdf-chunk-74 | Score: 0.520771
d from his duties for unknown reasons and failed to apply the vehicle's brakes, the automatic train control system was unable to detect the imminent collision and intervene. 4 Lessons Learned This collision could have been prevented by a train control system that did not depend solely on operator vigilance. A system that leaves the operator as a single point of failure is vulnerable to safety risks such as operator fatigue, distraction, or incapacitation. Requiring TBTC on rail transit systems,

Rank 4 | Source: passenger_alert_policy.txt | Chunk: passenger_alert_policy.txt-chunk-2 | Score: 0.516623
displays, the transit website, mobile notifications, and official social-media accounts. Priority 2 incidents require a public alert when the expected disruption exceeds ten minutes. Priority 3 incidents may be included in scheduled service notices.

Rank 5 | Source: incident_priority_policy.txt | Chunk: incident_priority_policy.txt-chunk-1 | Score: 0.512578
Municipal Transit Incident Priority Policy Priority 1 incidents include complete signal-system failure, collision, fire, evacuation, or any condition presenting an immediate threat to passenger safety. Priority 1 incidents must be escalated immediately to the operations control center and emergency responders. Priority 2 incidents include major delays, partial route suspension, station closure, and equipment failures that significantly affect service but do not create an immediate safety threat.

```

Figure 57. Q2 retrievals and answers at $k=1$, $k=3$, and the start of $k=5$.



```
Windows PowerShell
Context-RAG answer at k=5:
The signal failure with an immediate safety threat should be classified as an "Open/Acceptable Response" based on the recommendation in [Source 2]. Passengers should receive updates at least every fifteen minutes while the disruption remains active, as specified in [Source 1].

Saved required RAG outputs:
retrieved_chunks.json
configuration_comparison.json
k_sweep.json
rag_evaluation.json
rag_evaluation.csv
```

Figure 58. Completion of $k=5$ and saved machine-readable RAG outputs.

At $k=1$, the passenger-alert policy produces the clearest answer but does not independently supply the priority classification. At $k=3$, irrelevant collision-report text creates a false classification. At $k=5$, the required incident-priority policy appears, but ranking and filtering still allow distracting evidence to dominate. Thus, larger k improves raw source coverage but does not automatically improve the final context.

5.5 Evaluation

```

code > hw04 > rag.py > evaluate
'''
273 def evaluate(question, configuration, answer, retrieval):
274     expected_sources = set(question["expected_sources"])
275     retrieved_sources = {
276         result["source"]
277         for result in retrieval
278     }
279
280     refusal_needed = question["id"] in {"Q5", "Q6"}
281     refused = REFUSAL.lower() in answer.lower()
282
283     keywords = expected_keywords(question["id"])
284
285     if refusal_needed:
286         correct_answer = refused
287     elif keywords:
288         correct_answer = all(
289             keyword in answer.lower()
290             for keyword in keywords
291         )
292     else:
293         correct_answer = bool(answer.strip())
294
295     if expected_sources:
296         correct_retrieval = expected_sources.issubset(
297             retrieved_sources
298         )
299     else:
300         correct_retrieval = True
301
302     cited = bool(
303         re.search(r"\[Source \d+\]", answer)
304     )
305
306     grounded = (
307         correct_answer
308         and (
309             refused
310             if refusal_needed
311             else (
312                 configuration == "C"
313                 and correct_retrieval
314                 and cited
315             )
316         )
317     )
318
319     format_compliance = (
320         refused or cited
321         if configuration == "C"
322         else bool(answer.strip())
323     )
324
325     return {
326         "correct_retrieval": correct_retrieval,
327         "correct_answer": correct_answer,

```

Figure 59. Evaluation logic checks expected sources, answer keywords, citations, grounding, and refusal behavior.

```

Windows PowerShell
(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562> python .\code\hw04\summarize_rag.py 2>&1 |
>> Tee-Object -FilePath .\reports\hw04\RUN_LOG.txt -Append
HW4 RAG evaluation summary
A - No RAG: answer_accuracy=33.3%; faithfulness=0.0%; format=100.0%; refusal_robustness=0.0%
B - Basic RAG: answer_accuracy=50.0%; faithfulness=0.0%; format=100.0%; refusal_robustness=0.0%
C - Context RAG: answer_accuracy=83.3%; faithfulness=66.7%; format=83.3%; refusal_robustness=100.0%

Saved: D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw04\raw\rag_summary.json
Updated: D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw04\METRICS.md
(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562>

```

Figure 60. Corrected RAG summary reports Context RAG faithfulness of 66.7%.

Configuration	Retrieval accuracy	Answer accuracy	Faithfulness	Format compliance	Q5/Q6 refusal
A No RAG	33.3%	33.3%	0.0%	100.0%	0.0%
B Basic RAG	66.7%	50.0%	0.0%	100.0%	0.0%
C Context RAG	66.7%	83.3%	66.7%	83.3%	100.0%

5.6 RAG results analysis

The experiment compared No RAG, Basic RAG, and Context-Engineered RAG across six precommitted questions. Context RAG performed best, with 83.3% answer accuracy, 66.7% faithfulness, 83.3% format compliance, and 100% refusal robustness for Q5 and Q6. No RAG reached only 33.3% answer accuracy, while Basic RAG reached 50.0%. Q1 retrieved the passenger-alert policy and returned the required five-minute timing. The retrieved chunk directly contained the answer, so both retrieval-augmented configurations improved factual precision over the baseline. Q2 exposed the main retrieval failure. The answer required both the incident-priority policy and passenger-alert policy, but top-k 3 retrieved the alert policy and two collision-report chunks. The irrelevant “Open-Acceptable Response” text then contaminated the classification. This was primarily a retrieval and context-selection failure, although the generator also failed to reject the misleading phrase. Q3 retrieved relevant VTA safety material, but the selected evidence did not include both expected documents. Q4 correctly showed that the required response depends on incident priority, although Context RAG added the refusal sentence after providing a supported answer, reducing format compliance. The k sweep showed why more context is not always better. At k=1, the answer was concise but the single chunk did not fully support both requested facts. At k=3, irrelevant collision text produced the wrong classification. At k=5, the missing priority policy finally appeared, but it ranked below distracting chunks and did not control the generated answer. The best raw coverage was k=5, while k=1 produced the cleanest answer for this test. Q5 and Q6 demonstrated the strongest benefit of grounding rules. The baseline speculated about a future fare and invented a World Series winner. Context RAG returned the exact required refusal for both unsupported questions. Overall, source metadata and ordering made evidence traceable, de-duplication and score filtering reduced some noise, and the refusal rule sharply reduced hallucination. However, the Q2 and Q3 results show that retrieval quality still limits answer quality: a grounded prompt cannot cite evidence that the retriever fails to select.

6 Verification and Reproducibility

The self-check script tests objective behavior without modifying application code. It verifies FastAPI availability, session creation, naive and optimized endpoints, the required database variable and row counts, all 180 measurements, the five-document corpus, the six top-k retrievals, the three configurations, the k sweep, exact Q5/Q6 refusals, and all 18 evaluation rows.

```

code > verify_hw04.py > verify_api
128 def verify_api():
129     started_here = start_server()
130     add_check(
131         "FastAPI responds on PORT_BASE",
132         server_responds(),
133         f"GET {BASE_URL}/docs returned successfully"
134     )
135
136     cookie_jar = http.cookiejar.CookieJar()
137     opener = urllib.request.build_opener(
138         urllib.request.HTTPCookieProcessor(cookie_jar)
139     )
140
141     status, login = request_json(
142         opener,
143         "POST",
144         "/api/login",
145         {
146             "email": "student@example.com",
147             "password": "Transit2562!"
148         }
149     )
150
151     add_check(
152         "Login creates a server-side session",
153         status == 200 and len(cookie_jar) > 0,
154         (
155             f"status={status}; "
156             f"http_only_session_cookie_received="
157             f"{len(cookie_jar) > 0}"
158         )
159     )
160
161     naive_status, naive = request_json(
162         opener,
163         "GET",
164         "/api/performance/incidents-naive?page_size=10"
165     )
166
167     fixed_status, fixed = request_json(
168         opener,
169         "GET",
170         "/api/performance/incidents-optimized?page_size=10"
171     )
172

```

Figure 61. Self-check code starts the API, logs in, and invokes both performance endpoints.

```

verify_hw04.py > verify_api
def verify_api():
    naive_queries = (
        naive.get("sql_queries")
        if isinstance(naive, dict)
        else None
    )
    fixed_queries = (
        fixed.get("sql_queries")
        if isinstance(fixed, dict)
        else None
    )

    add_check(
        "Naive list endpoint returns related data",
        (
            naive_status == 200
            and count_records(naive) == 10
            and naive_queries == 11
        ),
        (
            f"status={naive_status}; "
            f"records={count_records(naive)}; "
            f"sql_queries={naive_queries}"
        )
    )

    add_check(
        "Optimized list endpoint returns related data",
        (
            fixed_status == 200
            and count_records(fixed) == 10
            and fixed_queries == 1
        ),
        (
            f"status={fixed_status}; "
            f"records={count_records(fixed)}; "
            f"sql_queries={fixed_queries}"
        )
    )

    return started_here

```

Figure 62. Self-check assertions require 10 records, 11 naive queries, and 1 optimized query.

```

code > verify_hw04.py > main
394 def main():
414     result = {
415         "homework_number": 4,
416         "sid4": 2562,
417         "commit_hash": commit_hash(),
418         "model_configuration": {
419             "llm": "qwen3:1.7b",
420             "embedding_model": (
421                 "sentence-transformers/all-MiniLM-L6-v2"
422             ),
423             "device": "CPU",
424             "chunk_size": 500,
425             "chunk_overlap": 50,
426             "default_top_k": 3,
427             "port_base": PORT,
428             "database": "s2562_rel"
429         },
430         "seed": 2562,
431         "verify_seed": 262562,
432         "verified_at_utc": datetime.now(
433             timezone.utc
434         ).isoformat(),
435         "checks": checks,
436         "overall_passed": (
437             bool(checks)
438             and all(check["passed"] for check in checks)
439         )
440     }
441
442     OUTPUT_FILE.write_text(
443         json.dumps(result, indent=2),
444         encoding="utf-8"
445     )
446
447     print(json.dumps(result, indent=2))
448
449     raise SystemExit(
450         0 if result["overall_passed"] else 1
451     )
452
453
454 if __name__ == "__main__":
455     main()

```

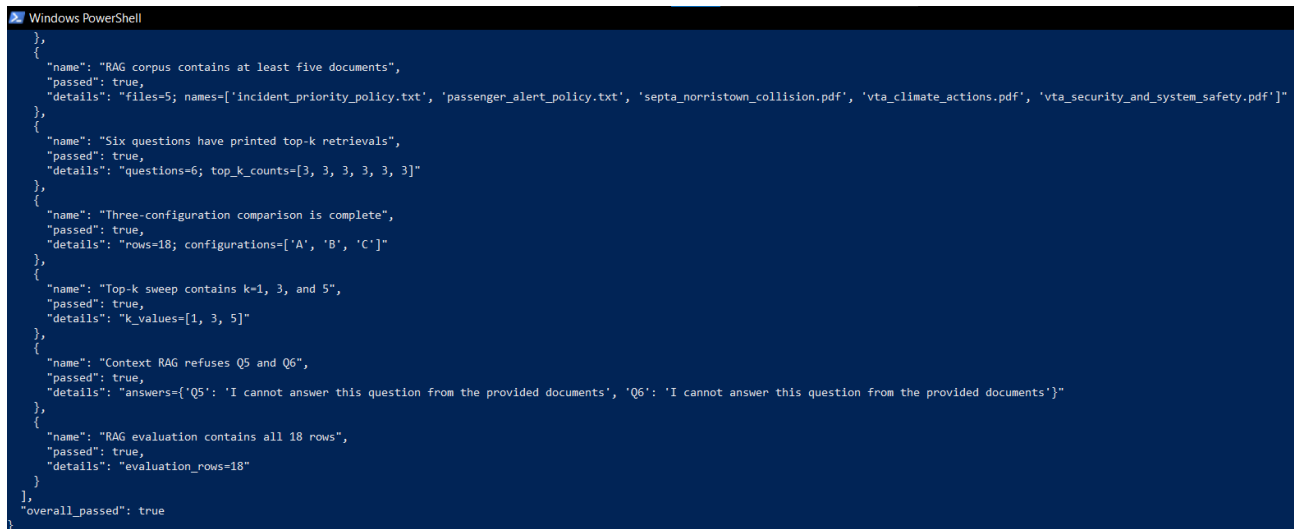
Figure 63. verification.json metadata includes homework, SID4, commit, model configuration, seeds, checks, and overall status.

```

Windows PowerShell
(.venv) PS D:\Grad Prep\STSU\GitHub\data260-2562> python .\code\verify_hw04.py
{
  "homework_number": 4,
  "sid4": 2562,
  "commit_hash": "5a5077d8cfa24823fba3d75f0e88cb0fd3aaffca",
  "model_configuration": {
    "llm": "qwen3:1.7b",
    "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
    "device": "CPU",
    "chunk_size": 500,
    "chunk_overlap": 50,
    "default_top_k": 3,
    "port_base": 8762,
    "database": "s2562_rel"
  },
  "seed": 2562,
  "verify_seed": 262562,
  "verified_at_utc": "2026-10-01T04:39:21.114321+00:00",
  "checks": [
    {
      "name": "FastAPI responds on PORT_BASE",
      "passed": true,
      "details": "GET http://127.0.0.1:8762/docs returned successfully"
    },
    {
      "name": "Login creates a server-side session",
      "passed": true,
      "details": "status=200; http_only_session_cookie_received=True"
    },
    {
      "name": "Naive list endpoint returns related data",
      "passed": true,
      "details": "status=200; records=10; sql_queries=11"
    },
    {
      "name": "Optimized list endpoint returns related data",
      "passed": true,
      "details": "status=200; records=10; sql_queries=1"
    },
    {
      "name": "Required database session variable exists",
      "passed": true,
      "details": "Checked code.hw04.backend.database"
    },
    {
      "name": "MySQL seed data exists",
      "passed": true,
      "details": "incidents=5000; incident_notes=200"
    },
    {
      "name": "All 180 N+1 requests are saved",
      "passed": true,
      "details": "rows=180; groups=({'naive', 10): 30, ('optimized', 50): 30, ('naive', 200): 30, ('optimized', 10): 30, ('optimized', 200): 30, ('naive', 50): 30}"
    }
  ],
}

```

Figure 64. Verification output confirms API, authentication, database, seed, and N+1 checks passed.



```

{
  "name": "RAG corpus contains at least five documents",
  "passed": true,
  "details": "files=5; names=['incident_priority_policy.txt', 'passenger_alert_policy.txt', 'septa_norristown_collision.pdf', 'vta_climate_actions.pdf', 'vta_security_and_system_safety.pdf']"
},
{
  "name": "Six questions have printed top-k retrievals",
  "passed": true,
  "details": "questions=6; top_k_counts=[3, 3, 3, 3, 3, 3]"
},
{
  "name": "Three-configuration comparison is complete",
  "passed": true,
  "details": "rows=18; configurations=['A', 'B', 'C']"
},
{
  "name": "Top-k sweep contains k=1, 3, and 5",
  "passed": true,
  "details": "k_values=[1, 3, 5]"
},
{
  "name": "Context RAG refuses Q5 and Q6",
  "passed": true,
  "details": "answers={'Q5': 'I cannot answer this question from the provided documents', 'Q6': 'I cannot answer this question from the provided documents'}"
},
{
  "name": "RAG evaluation contains all 18 rows",
  "passed": true,
  "details": "evaluation_rows=18"
}
},
"overall_passed": true
}

```

Figure 65. Verification output confirms all RAG checks and overall passed true.

The verification run recorded commit 5a5077d8cfa24823fba3d75f0e88cb0fd3aaffca. The report header records the later submission commit. The generated reports/hw04/verification.json should be committed before creating the final hw4 tag.

7 AI Use Disclosure

7.1 What did I use an AI assistant for, and what did I do myself?

I used ChatGPT to troubleshoot environment setup, package installation, PowerShell commands, and Python module invocation errors. I independently developed and ran the React, FastAPI, MySQL, N+1, and RAG work; reviewed the outputs; verified the generated files; captured the screenshots; and interpreted the results.

7.2 What AI-produced output was wrong or unsuitable?

An early command ran the HW4 seed module with the system Python interpreter. That environment did not contain SQLAlchemy, so the command was unsuitable for the project environment and failed with `ModuleNotFoundError: No module named sqlalchemy`.

7.3 How did I detect the problem or verify the result?

The missing-package traceback appeared immediately, and the PowerShell prompt did not show the active virtual environment. I compared the interpreter context with the project setup and then confirmed that the required packages were installed inside `.venv`.

7.4 What did I change, and why does it work now?

I activated `.venv`, ran the command from the repository root, and used the package module path so Python could resolve the relative imports and installed dependencies. The seed script then completed, and MySQL Workbench confirmed 5,000 incidents and 200 related notes.

8 Conclusion

HW4 is complete across the four required parts. The React client performs authenticated CRUD through an HTTP-only server-side session, MySQL persists the domain and session data, the optimized list endpoint removes the N+1 query growth, and the grounded RAG configuration substantially improves answer accuracy and refusal behavior. The verification script reports `overall_passed true`, and the report includes code and corresponding output evidence for each required component.